Unit VI

6 Planning

6.1 : Automated Planning, Classical Planning, Algorithms for Classical Planning, Heuristics for Planning

Q.1 Explain the three components of representing the actions in classical planning with example.

Ans. : Representing actions :

1) Action is represented using two components.
   i) Preconditions - That should hold before the action can be executed.
   ii) Effects - Are the outputs that will come after executing the action.
2) The representation of the action is collectively called as action schema which has three parts :
   i) The action name and the parameter list that serves to identify the action.
   ii) The precondition - Which is a conjunction of function-free positive literals. Any variables in the precondition must appear in actions parameter list.
   iii) The effect - It is a conjunction of function-free literals describing, how the state changes, when the action is executed.

   A positive literal P in the effect is true in the state resulting from the action, where as, negative literal ¬ P is false in the effect for dividing the effect. Some planning system use add list for positive literals and delete list for negative literals.

3) In Strips every literal which remains unchanged after action, is not mentioned in the effect. This saves considerable amount of memory thereby avoiding frame problem.

Q.2 Explain air cargo transport example using STRIPS language.

Ans. : Air cargo transport example using STRIPS :

⇒ Actions

1) Load (cargo, plane, airport)
2) Unload (cargo, plane, airport)
3) Fly (plane, airport, airport)

⇒ Predicates

1) In (Cargo, Plane)
2) At (Cargo ∨ plane, airport)

⇒ Problem solution

Load (C1, P1, SFO), Fly (P1, SFO, JFK),
Unload (C1, P1, JFK)
Load (C2, P2, JFK), Fly (P2, JFK, SFO)
Unload (C2, P2, SFO)

where,
C1, C2 are Cargo
P1, P2 are Plane
SFO, JFK are airport names.

Q.3 Solve the blocks world problem using forward state-space search and backward state-space search.

Ans. : The blocks world

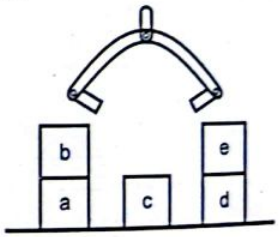


Fig. Q.3.1

Represent this world using predicates

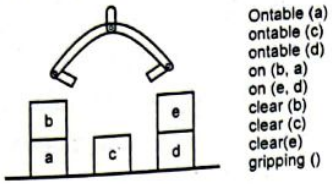


Fig. Q.3.2

The robot arm can perform these tasks

- pickup (W) : Pick up block W from its current location on the table and hold it.
- putdown (W) : Place block W on the table.
- stack (U, V) : Place block U on top of block V.
- unstack(U, V) : Remove block U from the top of block V and hold it.
- All assume that the robot arm can precisely reach the block.

Portion of the search space of the blocks word example

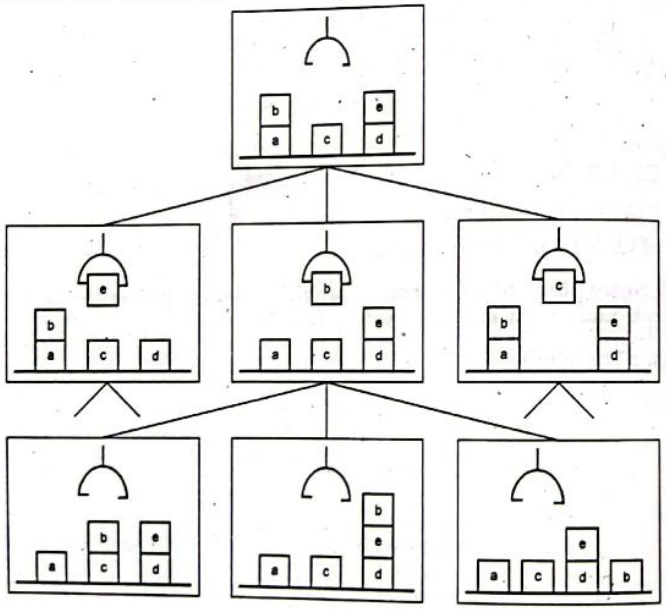


Fig. Q.3.3

Four operators for the blocks world

pickup(X)	P : gripping() ∧ clear(X) ∧ ontable(X)
	A : gripping (X)
	D : ontable(X) ∧ gripping()
putdown(X)	P : gripping(X)
	A : ontable(X) ∧ gripping() ∧ clear(X)
	D : gripping(X)
stack(X,Y)	P : gripping(X) ∧ clear(Y)
	A : on(X,Y) ∧ gripping() ∧ clear(X)
	D : gripping(X) ∧ clear(Y)
unstack(X,Y)	P : gripping() ∧ clear(X) ∧ on(X,Y)
	A : gripping(X) ∧ clear(Y)
	D : on(X,Y) ∧ gripping()

The STRIPS representation

Special purpose representation.

An operator is defined in terms of its :

name,
parameters,
preconditions, and
results.

A planner is a special purpose algorithm, i.e., it's not a general purpose logic theorem prover.

Notice the simplification

Preconditions, add lists and delete lists are all conjunctions. We don't have the full power of predicate logic.

The same applies to goals. Goals are conjunctions of predicates.

A detail : Why do we have two operators for picking up (pickup and unstack), and two for putting down (putdown and stack)?

A goal state for the block world

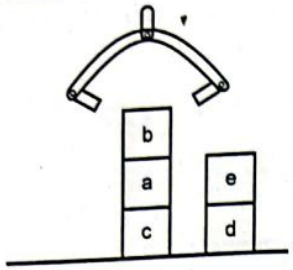


Fig. Q.3.4

Forward chaining

Initial :

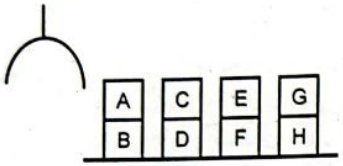


Fig. Q.3.5

Goal :

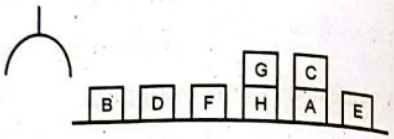


Fig. Q.3.6

1st and 2nd levels of search

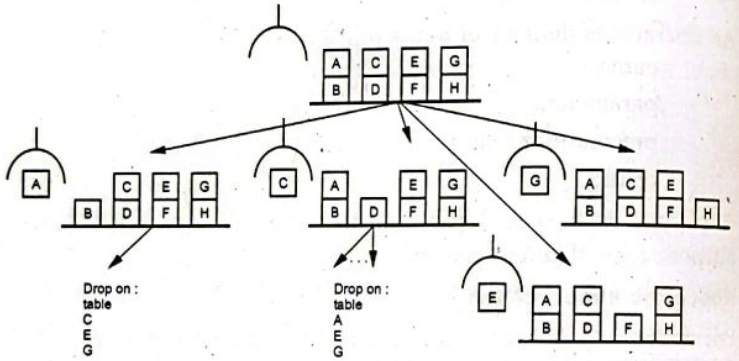


Fig. Q.3.7

Results

A plan is :

- unstack(A, B)
- putdown (A)

- unstack(C, D)
- stack (C, A)
- unstack(E, F)
- putdown (F)
- Notice that the final locations of D, F, G and H need not be specified
- Also notice that D, F, G and H will never need to be moved. But there are states in the search space which are a result of moving these. Working backwards from the goal might help.

Backward chaining

Initial :

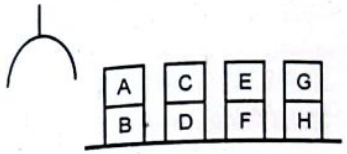


Fig. Q.3.8

Goal :

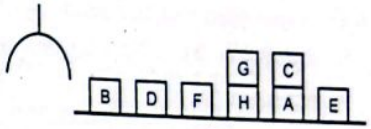


Fig. Q.3.9

1st level of search

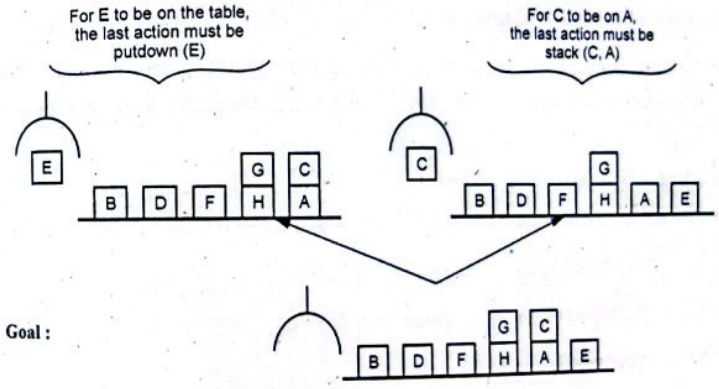


Fig. Q.3.10

2nd level of search

where was E picked up from?

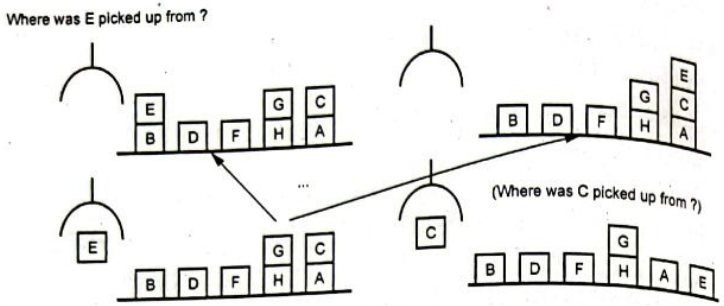


Fig. Q.3.11

Results

- The same plan can be found
  - unstack(A, B)
  - putdown (A)
  - unstack(C, D)
  - stack (C, A)
  - unstack(E, F)
  - putdown (F)
- Now, the final locations of D, F, G and H need to be specified.
- Notice that D, F, G and H will never need to be moved. But observe that from the second level on the branching factor is still high.

Partial-order planning (POP)

- Notice that the resulting plan has two parallelizable threads :

unstack(A,B)	unstack(E, F)
putdown (A)	putdown (F)
unstack(C,D)	
stack (C,A)	

- These steps can be interleaved in three different ways :

unstack(E, F)	unstack(A,B)	unstack(A,B)
putdown (F)	putdown (A)	putdown (A)
unstack(A,B)	unstack(E, F)	**unstack(C,D)**
putdown (A)	putdown (F)	**stack (C,A)**
unstack(C,D)	**unstack(C,D)**	unstack(E, F)
stack (C,A)	**stack (C,A)**	putdown (F)

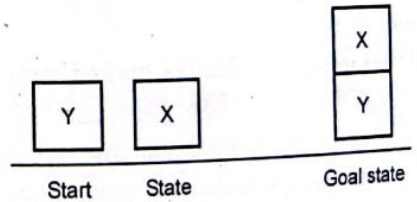
Q.4 Explain how planning problem is expressed in STRIPS. Use air cargo transport as an example.

Ans. : Planning using STRIPS : In this approach, each operation is described by a list of new predicates that cause operator to be true and list of old predicates that it causes to beome false. According to strips each operator has three lists of predicates associated with it.

- A list of things that become TRUE called ADD list.
- A list of things that become FALSE called DELETE list.
- A set of prerequisites or preconditions that must be true before the operator can be applied.
- Anything not in these lists is assumed to be unaffected by the operation. Consider the following example in the blocks world and the fundamental operations.

START : ONTABLE (X) ∧ ONTABLE (Y) ∧ CLEAR (X) ∧ CLEAR (Y) ∧ AREMPTY

GOAL : ON (X, Y) ∧ CLEAR (X) ∧ AREMPTY



Air cargo transport example using STRIPS :

⇒ Actions

1) Load (cargo, plane, airport)

2) Unload (cargo, plane, airport)

3) Fly (plane, airport, airport)

⇒ Predicates

1) In (Cargo, Plane)

2) At (Cargo / plane, airport)

⇒ Problem solution

Load (C1, P1, SFO), Fly (P1, SFO, JFK),
Unload (C1, P1, JFK)
Load (C2, P2, JFK), Fly (P2, JFK, SFO)
Unload (C2, P2, SFO)

where

C1, C2 are Cargo

P1, P2 are Plane

SFO, JFK are airport names

Q.5 Explain the basic representation for planning.

Ans. : Representation of Planning Problem : For representing planning problem we need to represent states, goals and actions.

• Representing states

1. World is decomposed into logical conditions representing various states of the world.
2. State is represented as a conjunction of positive literals using first order logic.
3. Literals in first order logic should be ground terms (i.e. without any function).
4. Any conditions about the world that are not mentioned are assumed to be false (i.e. a closed world assumption is used).

For example :

Correct	Wrong
At (Ram, Janki) ∧	At (Son of Dashrath,
At (Ram, Ayodhya)	Ayodhya)
	(Functions not allowed)

• Representing goals

1. It is a partially specified state.
2. It is represented as conjunction of positive ground literals.
3. A propositional state 's' satisfies a goal 'g' if 's' contains all the atoms of 'g' (s may contain more atoms).

For example : The statement

Batman ∧ Rookie ∧ All rounder satisfies the goal Batman ∧ Rookie.

• Representing actions

1) Action is represented using two components.
   i) Preconditions : that should hold before the action can be executed.
   ii) Effects : are the outputs that will come after executing the action.
2) The representation of the action is collectively called as **action schema** which has three parts :
   i) The action name and the parameter list that serve to identify the action.
   ii) The precondition : Which is a conjunction of function-free positive literals. Any variables in the precondition must appear in action parameter list.
   iii) The effect : It is a conjunction of function-free literals describing how the state changes when the action is executed.

   A positive literal 'P' in the effect is true in the state resulting from the action, where as negative literal '¬P' is false in the

effect for dividing the effect. Some planning system use add list for positive literals and delete list for negative literals.

3) In Strips every literal which remains unchanged after action, is not mentioned in the effect. This saves considerable amount of memory thereby avoiding frame problem.

- **Representation of a solution :** Solution to a planning problem is action sequence. When this action sequence is executed in the initial state, its execution results in a state that satisfies the goal.

Q.6 What is partial-order planning ? Explain with suitable example.

Ans. : Total order planner do not consider problem decompositions which could help to get early solution. So we need the planner that works on several subgoals and then combines the subplans. In this planner, important decisions comes first, rather than being forced to work on steps in some fixed predefined chronological order. Such planners are partial ordered planner.

The planning algorithm that can place two actions into a plan without specifying which comes first is called as **partial order planner.**

In this, problem is decomposed into several subproblems thereby generating various subgoals.

- Each subgoal is solved independently, once its action sequence is generated.
- All the subgoals, action sequences are then combined to solve complete problem. That is a action sequence is generated for problem as a whole.
- A partial order planner can generate various solution sequences through different combinations of subsequence solutions. Each of this final solution is called as **linearization** of the partial-order plan.
- Every linearization of partial-order solution is a total-order solution whose execution from the initial state will reach a goal state.

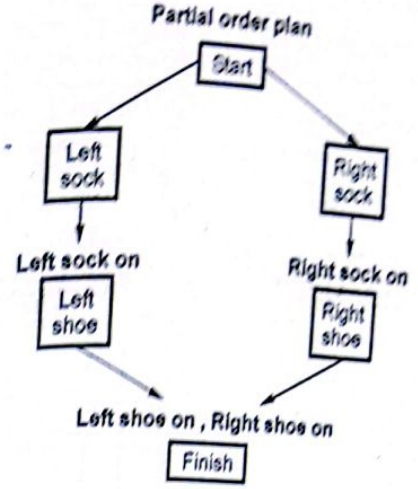


Fig. Q.6.1 A partial-order plan for putting on shoes and socks

For example - Partial Order Planner -

Problem statement :- Putting on a pair of shoes.

Representation of the problem as formal planning problem -

Goal (RightShoeOn ∧ LeftShoeOn)

Init ( )

Action (RightSockOn, PRECOND : RightSockOn, EFFECT : RightSockOn)

Action(RightSockOn, EFFECT : RightSockOn)

Action (LeftSockOn, PRECOND : LeftSockOn, EFFECT : LeftSockOn)

Action (LeftSockOn, EFFECT : LeftSockOn).

Implementation of partial order planner :

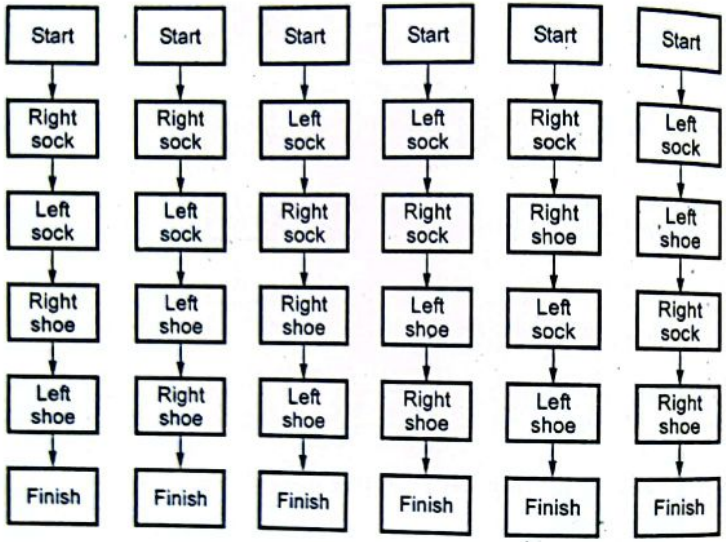


Fig. Q.6.2 The six corresponding linearizations into total order plans

Q.7 Explain any one classical planning approach in detail.

Ans. : Classical planning approach :

Goal stack planning :

- In this method, the problem solver makes use of a single stack that contains both goals and operators that have been proposed to satisfy those goals.
- The problem solver also relies on a database that describes the current situation and a set of operators described as PRECONDITION, ADD and DELETE lists.
- The goal stack planning method attacks problems involving conjoined goals by solving the goals one at a time, in order.
- A plan generated by this method contains a sequence of operators for attaining the first goal, followed by a complete sequence for the second goal etc.

- At each succeeding step of the problem solving process, the top goal on the stack will be pursued.
- When a sequence of operators that satisfies it is found, that sequence is applied to the state description, yielding new description.
- Next, the goal that is then at the top of the stack is explored and an attempt is made to satisfy it, starting from the situation that was produced as a result of satisfying the first goal.
- This process continues until the goal stack is empty.
- Then as one last check, the original goal is compared to the final state derived from the application of the chosen operators.
- If any components of the goal are not satisfied in that state, then those unsolved parts of the goal are reinserted onto the stack and the process is resumed.

Q.8 Explain planning graphs with suitable example.

Ans. : Planning Graphs :

1. Planning graphs are pictorial and effective tool for solving hard planning problems.
2. They are the data structures that give good heuristic estimates and can be applied to any searching techniques.
3. They can give a solution directly using specialized algorithm GRAPHPLAN.
4. Planning graphs work only for propositional planning problems, that is propositions with no variables.

Structure of Planning Graph :

1. Planning graph consists of a sequence of levels that correspond to time steps in the plan.
2. Level 0 is the initial state.
3. Each level contains a set of literals and set of actions.

4. Planning graph records only a restricted subset of the possible negative intractions among actions. Therefore it expects to have minimum number of steps for a literal to become true.
5. At each level, literals are all those that are true at that time step, which depends on previously executed action.
6. At each level, actions are all those actions, that could have their preconditions satisfied at that time step, which depends on which literals are true.
7. The number of steps in planning graph provides a good estimate of how difficult it is to achieve a given literal from the initial state.

Q.9 What is classical planning ? Explain the algorithm for planning state space search and backward relevant state search.

Ans. : Planning : Planning is the process of deciding the sequence of actions that will achieve the goal. In this chapter we will consider various planning techniques for AI agent who is having working environments that are fully observable, deterministic, finite, static (that is change happens only when the agent acts), discrete (in time, action, objects and effects). Such environments are called as classical planning environments.

Difficulties in planning problem : There are various reasons because of which handling the planning problem becomes difficult.

First reason is impact of irrelevant actions. The irrelevant actions unnecessarily can put agent in infinite loops while searching for required action. Irrelevant actions are the actions which are not just incorrect actions but those actions which make no sense.

Second reason is for choosing better actions we need good heuristic function. Designing of a good heuristic depends on problem. Therefore for every new problem, heuristic should be designed. AI agent lacks autonomy hence human needs to put heuristic function, every time, in AI agent.

Third reason is directly associated with characteristic of a problem. If problem is decomposable into various subproblem it become

easy to achieve the goal. But agent should be able to understand this decomposition attribute of the problem.

We can represent planning process as a problem. For its representation there are three requirements, (1) States (2) Actions (3) Goals. Once representation is done then planning algorithms can be deviced which will generate plan.

For representing the planning problem we need a language that is expressive and can describe variety of problems. The language should have proper syntax so that algorithms can operate on it efficiently.

There are various languages that can represent classical planner problems. The languages like Ada, Strips are used for representing the planning problem.

Algorithm for searching in the state-space : As there are no function symbols in description of planning problem the state space of planning problem is finite.

Therefore graph-search algorithm would be a complete algorithm for searching in state-space.

For example : A* Algorithm

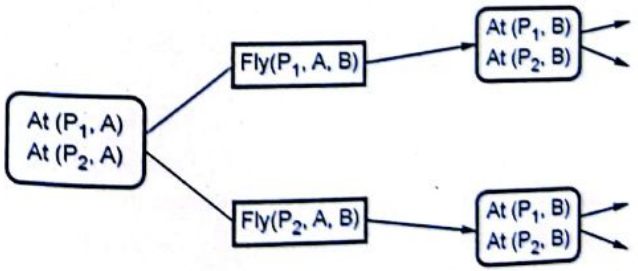


Fig. Q.9.1

Algorithm for backward search :

1. Any standard search algorithm like Breadth First Search can be used for backward search.

2. Algorithm terminates when a predecessor description is satisfied by the initial state of the planning problem.

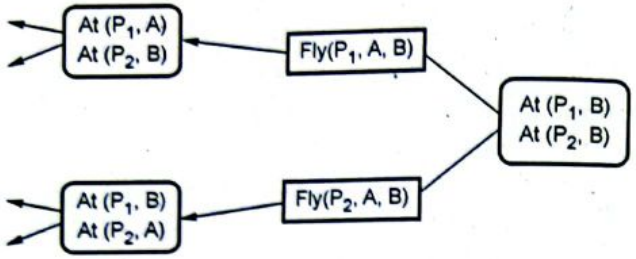


Fig. Q.9.2 Partial backward state-space search

Q.10 Explain planning as refinement search.

Ans. : Viewing planning as a refinement search provides a unified framework within which the complete solution of plan-space planning algorithms can be effectively cast and compared.

Whatever the exact nature of the planner, the ultimate aim of (classical) planning is to find a ground operator sequence , which when executed in the given initial state, will produce desired behaviors or sequences of world states. Most classical planning techniques have traditionally concentrated on the sub-class of behavioral constraints called the goals of attainment , which essentially constrain the agent's attention to behaviors that end in world states satisfying desired properties. For the most part, this is the class of goals. Same framework can be easily extended to a richer class of goals. The operators (aka actions) in classical planning are modeled as general state transformation functions.

Any potential solution for a planning problem must be a ground operator sequence. Thus, viewed as a refinement search, the candidate space, K , of a planning problem, is the set of all ground operator sequences. As an example, if the domain contains three ground actions a1, a2 and a3, then the regular expression

{a1,a2,a3} would describe the candidate space for this domain. A ground operator sequence is considered a solution to a planning problem in classical planning.

Q.11 Discuss simple planning agent.

Ans. :

- Take example of an agent which can be a coffee maker, a printer and a mailing system, also assume that there are 3 people who have access to this agent.
- Suppose at same time if all 3 users of an agent give a command to execute 3 different tasks of coffee making, printing and sending a mail.
- Then as per definition of planning, agent has to decide the sequence of these actions.
- Fig. Q.11.2 depicts a general diagrammatic representation of a planning agent that interacts with environment with its sensors and effectors/actuators. When a task comes to this agent it has to decide the sequence of actions to be taken and then accordingly execute these actions.

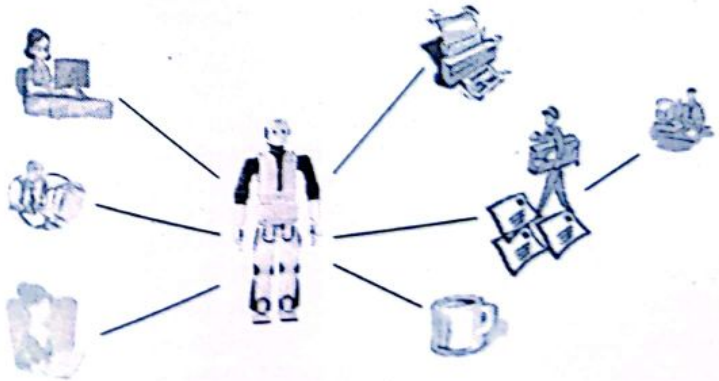


Fig. Q.11.1 : Example of a planning problem

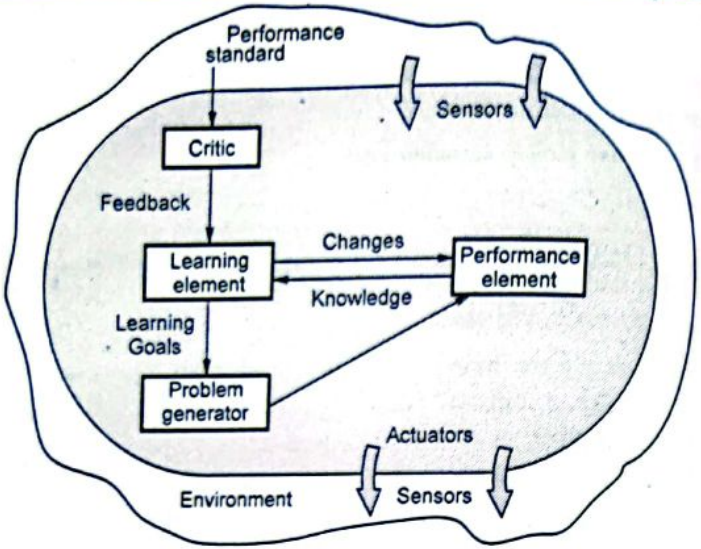


Fig. Q.11.2 Learning agent

Q.12 Explain planning problem.

Ans. : • For formulating planning problem there is a need of information and it is required to have list of expected results. Also it is understandable here that, states of an agent correspond to the probable surrounding environments while the actions and goals of an agent are specified based on logical formalization.

- To achieve any goal an agent has to answer few questions like "what will be the effect of its actions", "how it will affect the upcoming actions", etc. This illustrates that an agent must be able to provide a proper reasoning about its future actions, states of surrounding environments, etc.
- Consider simple Tic-Tac-Toe game. A Player cannot win a game in one step, he/she has to follow sequence of actions to win the game. While taking every next step he/she has to consider old steps and has to imagine the probable future actions of an opponent and accordingly make the next move and at the same time he/she should also consider the consequences of his/her actions.

A classical planning has the following assumptions about the task environment :

- **Fully Observable :** Agent can observe the current state of the environment.
- **Deterministic :** Agent can determine the consequences of its actions.
- Finite: There are finite set of actions which can be carried out by the agent at every state in order to achieve the goal.
- Static : Events are steady. External event which cannot be handled by agent is not considered.
- **Discrete :** Events of the agent are distinct from starting step to the ending(goal) state in terms of time.
- So, basically a planning problem finds the sequence of actions to accomplish the goal based on the above assumptions.
- Also note that, goal can be specified as a union of sub-goals.
- Take example of ping pong game where, points are assigned to opponent player when a player fails to return the ball within the rules of ping-pong game. There can a best 3 of 5 matches where, to win a match you have to win 3 games and in every game you have to win with a minimum margin of 2 points.

Q.13 How problem solving differs from planning.

OR Compare problem solving agent and planning agent.

Ans. : • Generally problem solving and planning methodologies can solve similar type of problems.

- Main difference between problem solving and planning is that planning is a more open process and agents follow logic - based representation.
- Planning is supposed to be more powerful than problem solving because of these two reasons.
- Planning agent has situations (i.e. states), goals (i.e. target end conditions) and operations (i.e. actions performed). All these parameters are decomposed into sets of sentences and further in sets words depending on the need of the system.

- Planning agents can deal with situations / states more efficiently because of its explicit reasoning capability also it can communicate with the world.
- Agents can reflect on their targets and we can minimize the complexity of the planning problem by independently planning for sub - goals of an agent.
- Agents have information about past actions, presents actions and the important point is that it can predict the effect of actions by inspecting the operations.
- Planning is a logical representation, based on situation, goals and operations, of problem solving.

Planning = Problem solving + Logical representation

Q.14 What is goal of planning ?

OR Explain goal of planning with super market example.

Ans. : • We plan activities in order to achieve some goals. Main goal can be divided into sub - goals to make planning more efficient.

- Take example of a grocery shopping at supermarket, suppose one wants to buy milk, bread and egg from supermarket, then your initial state will be - "at home" and goal state will be - "get milk, bread and egg".
- Now if one looks at the Fig. Q.14.1 one will understand that branching factor can be enormous depending upon the set of actions, for e.g. Watch TV, read book, etc., available at that point of time.
- Thus **branchingfactor** can be defined as a set of all probable actions at any state, set can be very large, such as in the supermarket example or block problem. If the domain of probable actions increases, the branching factor will also increase as they are directly proportional to each other, this will result in the increase of search space.

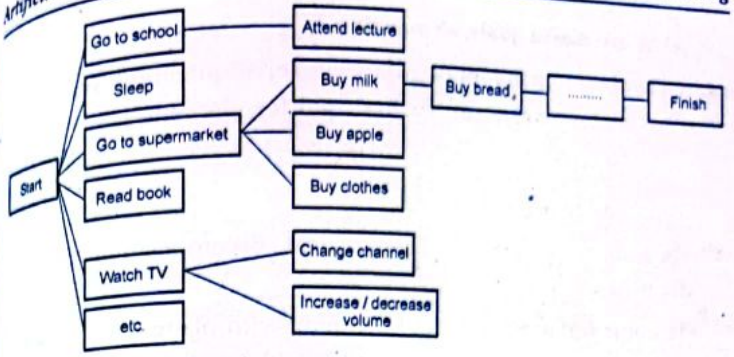


Fig. Q.14.1 : Supermarket examples to understand need of planning

- To reach the goal state you have to follow many steps, if you consider using heuristic functions, then you have to remember that it will not be able to eliminate states; these functions will be helpful only for guiding the search of states.
- So it becomes difficult to choose best actions. (i.e. even if we go to supermarket we need to make sure that all three listed items are picked, only then goal state can be achieved).
- As there are many possible actions and it is difficult to describe every state, and there can be combined goals (as seen in supermarket example) searching is inadequate to achieve goals efficiently. In order to be more efficient planning is required.
- In above sections, we discussed that planning requires explicit knowledge that means in case of planning we need to know the exact sequence of actions which will be useful in order to achieve the goal.
- Advantage of planning is that, the order of planning and the order of execution need not be same. For example, you can plan how to pay bills for grocery before planning to go to supermarket.
- Another advantage of planning is that you can make use of divide and conquer policy by dividing /decomposing the goal into sub goals.

Q.15 What are major goals of planning ?

Ans. : There are many approaches to solve planning problems. Following are few major **approaches** used for planning :

- Planning with state space search.
- Partial ordered planning.
- Hierarchical planning / hierarchical decomposition (HTN planning).
- Planning situation calculus / planning with operators.
- Conditional planning.
- Planning with operators.
- Planning with graphs.
- Planning with propositional logic.
- Planning reactive.

- Out of these major approaches we will be learning about following approaches in detail :
  - Planning with state space search
  - Partial ordered planning and
  - Hierarchical planning / Hierarchical decomposition (HTN planning).
  - Conditional planning.
  - Planning with operators.

Q.16 Analyze various planning approaches.

Ans. : • Planning combines the two major areas of AI namely, search and logic. A planner is a program that searches for a solution or one that proves the existence of a solution.

- Forward search addresses the problem heuristically. It finds patterns that can be seen as the independent sub - problems. Since this approach is heuristic, it can work even when the sub - problems are not completely independent.

- When the sub - goals are serializable and do not require any undo actions can achieve the goal efficiently.
- A planner which uses the bottom-up approach can solve any problem without backtracking but shortest plan cannot be guaranteed.
- The planners like as GRAPHPLAN or SATPLAN have enable the field of planning to reach upto a next level of planning.
- These techniques use heuristic approach through which the issues related to representation and combination are sorted. Although, there is a problem of scaling for such algorithms.
- In order to solve the large problems, these techniques just cannot rely on factored and propositional representation; but required to do synthesis of first order and hierarchical representations.

Q.17 Explain planning as state space search.

Ans. : Planning as state - space search

- It can be seen that an intelligent agent that can perform three tasks of printing, sending a mail and making coffee namely lets' call this agent as office agent.
- When this office agent gets order from three people at a same time to perform these three different tasks then, let us see how planning with state space search problem will look if we have a finite space.
- One can understand from Fig. Q.17.1 that the office agent is at location 250 on the state space grid. When he gets a task he has to decide which task can be performed more efficiently in lesser time.
- If it finds some input and output locations nearer on state space grid for example in case of printing task then the probability of performing that task will increase.

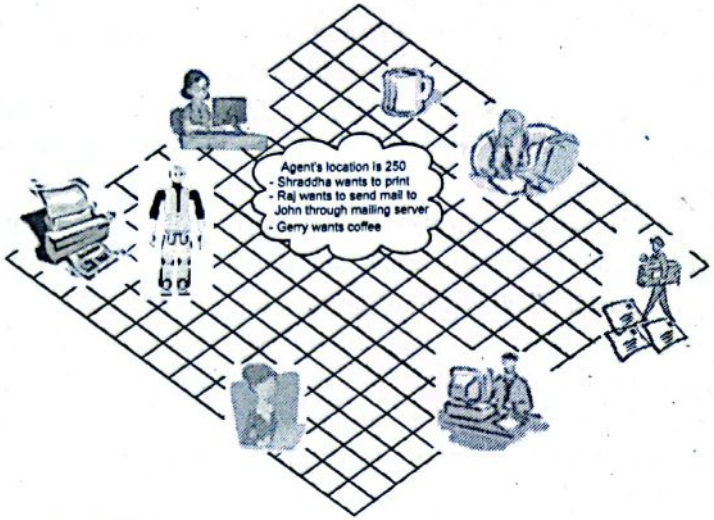


Fig. Q.17.1 : Office agent example with finite state space

- But to do this it should be aware of it's own current location, the locations of people who are assigning tasks and the locations of the required devices.
- State space search is unfavourable for solving real - world problems because, it requires complete description of every searched state, also search should be carried out locally.
- There can be two ways of representations for a state :

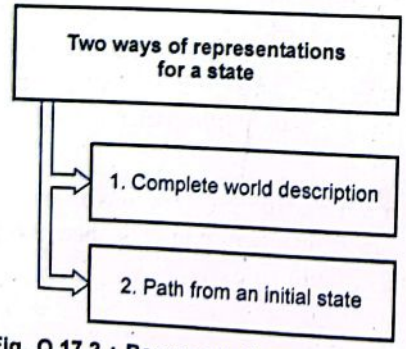


Fig. Q.17.2 : Representations for a state

1. Complete world description

- Description is available in terms of an assignment of a value to each previous suggestion.
- Or we can say that description is available as a suggestion that defines the state.
- Drawback of this types is that it requires a large amount of space.

2. Path from an initial state

- As per the name, path from an initial state gives the sequence of actions which are used to reach a state from an initial state.
- In this case, what holds in a state can be deduced from the axiom which specifies the effects of an actions.
- Drawback of these types is that it does not explicitly specify "What holds in every state".
- Because of this it can be difficult to determine whether two states are same.

Algorithms for planning as state space search

- As the name suggests state space search planning techniques is based on the spatial searching.
- Planning with state space search can be done by both forward and backward state - space search techniques.

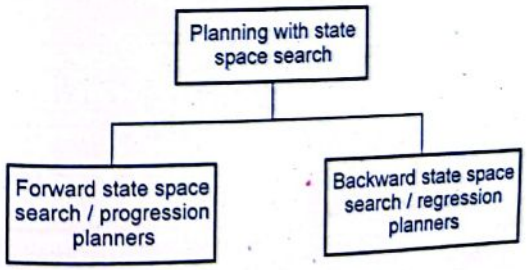


Fig. Q.17.3 : Classification of planning with state space search

Q.18 Explain progression planner.

Ans. : Progression Planners

- "Forward state - space search" is also called as **"progression planner".** It is a deterministic planning technique, as we plan sequence of actions starting from the initial state in order to attain the goal.
- With forward state space searching method we start with the initial state and go to final goal state. While doing this we need to consider the probable effects of the actions taken at every state.
- Thus the prerequisite for this type of planning is to have initial world state information, details of the available actions of the agent and description of the goal state.
- Remember that details of the available actions include preconditions and effects of that action.
- See Fig. Q.18.1 it gives a state - space graph of progression planner for a simple example where flight 1 is at location A and flight 2 is also at location A. These flights are moving from location A to location B. In 1st case only flight 1 moves from location A to location B, so the resulting state shows that after performing that action flight 1 is at location B whereas flight 2 is at its original location - A. similarly In 2nd case only flight 2 moves from location A to location B and the resulting state shows that after performing that action flight 2 is at location B while flight 1 is at its original location – A.

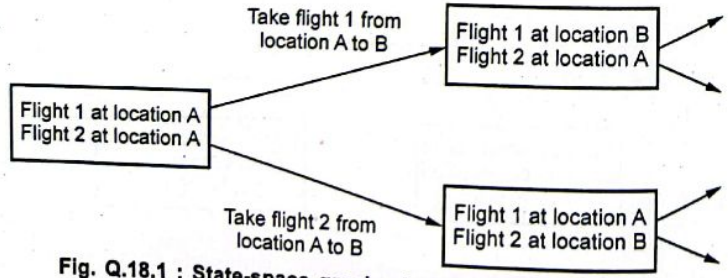


Fig. Q.18.1 : State-space graph of a progression planners

- It can be observed from the Fig. Q.18.1 that, rectangles show state of the flights (i.e. their current location) and lines give the corresponding actions from one state to another (i.e. move from one location to other location).
- Note that the lines coming out of every state matches to all of the permissible actions which can be accepted if the agent is in that state.

Progression planner algorithm

1. Formulize the state space search problem :
   - Initial state is the first state of the planning problem which has a set of positive, the literals which don't appear are considered as false.
   - If preconditions are satisfied then the actions are favoured i.e. if the preconditions are satisfied then positive effect literals are added for that action else the negative effect literals are deleted for that action.
   - Perform goal testing by checking if the state will satisfy the goal.
   - Lastly keep the step cost for each action as 1.
2. Consider example of A* algorithm. A complete graph search is considered as a complete planning algorithm. Functions are not used.
3. Progression planner algorithm is supposed to be inefficient because of the irrelevant action problem and requirement of good heuristics for efficient search.

Q.19 Explain regression planner.

Ans. : • "Backward state - space search" is also called as **"regression planner"** from the name of this method one can make out that the processing will start from the finishing state and then you will go backwards to the initial state.

- So basically we try to backtrack the scenario and find out the best possibility, in-order to achieve the goal to achieve this we have to see what might have been correct action at previous state.

- In forward state space search we used to need information about the successors of the current state now, for backward state - space search we will need information about the predecessors of the current state.
- Here the problem is that there can be many possible goal states which are equally acceptable. That is why this approach is not considered as a practical approach when there are large numbers of states which satisfy the goal.
- Let us see flight example, here you can see that the goal state is flight 1 is at location B and flight 2 is also at location B. We can see in Fig. Q.19.1 that if this state is checked backwards we have two acceptable states in one state only flight 2 is at location B, but flight 1 is at location A and similarly in 2nd possible state flight 1 is already at location B, but flight 2 is at location A.
- As we search backwards from goal state to initial state, we have to deal with partial information about the state, since we do not yet know what actions will get us to goal. This method is complex because we have to achieve a conjunction of goals.
- In this Fig. Q.19.1 rectangles are goals that must be achieved and lines shows the corresponding actions.

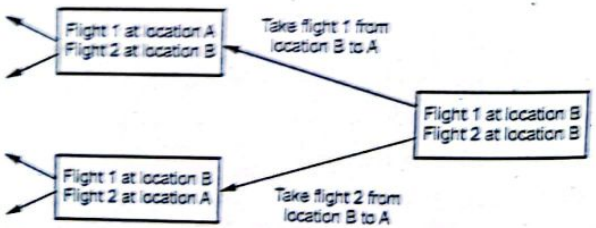


Fig. Q.19.1 : State-space graph of a regression planners

Regression algorithm

1. Firstly predecessors should be determined :
   - To do this we need to find out which states will lead to the goal state after applying some actions on it.

- We take conjunction of all such states and choose one action to achieve the goal state.
- If we say that "X" action is relevant action for first conjunction, only if pre-conditions are satisfied it works.
- Previous state is checked to see if the sub-goals are achieved.

2. Actions must be consistent it should not undo preferred literals. If there are positive effects of actions which appear in goal then they are deleted. Otherwise each precondition literal of action is added, except it already appears.
3. Main advantage of this method is only relevant actions are taken into consideration. Compared to forward search, backward search method has much lower branching factor.

Heuristics for state-space search

- Progression or Regression is not very efficient with complex problems. They need good heuristic to achieve better efficiency. Best solution is NP Hard (NP stands for Non-deterministic and Polynomial-time).
- There are two ways to make state space search efficient :
  - **Use linear method :** Add the steps which build on their immediate successors or predecessors.
  - **Use partial planning method :** As per the requirement at execution time ordering constraints are imposed on agent.

Q.20 Discuss total order planning.

Ans. : • Forward and regression planners impose a total ordering on actions at all stages of the planning process.

- In case of Total Order Planning (TOP), we have to follow sequence of actions for the entire task at once and to do this we can have multiple combinations of required actions. Here, we need to remember one most important thing which should be taken care of, is that TOP should take care of preconditions while creating the sequence of actions.

- For example, we cannot wear left shoe without wearing the left sock and we cannot wear the right shoe without wearing the right sock. So while creating the sequence of actions in total ordered planning, wearing left sock action should be executed before wearing the left shoe and wearing the right sock action should be executed before wearing the right shoe. As one can see in Fig. Q.20.1.

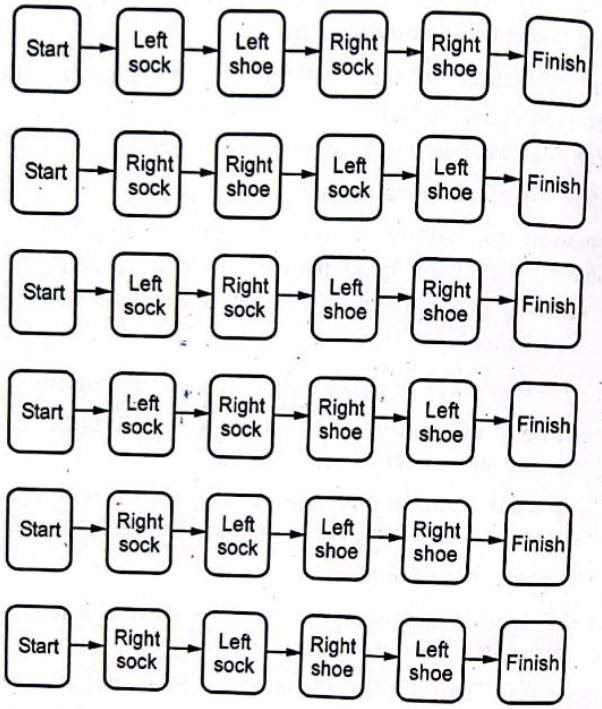


Fig. Q.20.1 : Total order planning of wearing shoe

- If there is a cycle of constraints then, total ordered planning cannot give good results. TOP can fail in non-cooperative environments. So we have Partial Ordered Planning method.

Q.21 Explain process of generating solution for partial order planning problem.

Ans. : • As consistent plan does not have cycle of constraints; it does not have conflicts in the causal links and does not have open preconditions so it can provide a solution for POP problem.

- While solving POP problem operators can add links and steps from existing plans to open preconditions in order to fulfil the open preconditions and then steps can be ordered with respect to the other steps for removing the potential conflicts. If the open precondition is unattainable, then backtrack the steps and try solving the problem with POP.
- Partial ordered planning is a more efficient method, because with the help of POP we can progress from vague plan to complete and correct solution in a faster way. Also we can solve a huge state space plan in less number of steps, this is because search takes place only when sub - plans interact.

Q.22 Explain planning with example.

Ans. :

1. First identify a hierarchy of major conditions.
2. Construct a plan in levels (Major steps then minor steps), so one can postpone the details to next level.
3. Patch major levels as detail actions become visible.
4. Finally demonstrate.

Example

- Actions required for "Travelling to Rajasthan" can be given as follows :
- Opening yatra.com (1)
- Finding train (2)
- Buy ticket (3)
- Get taxi(2)

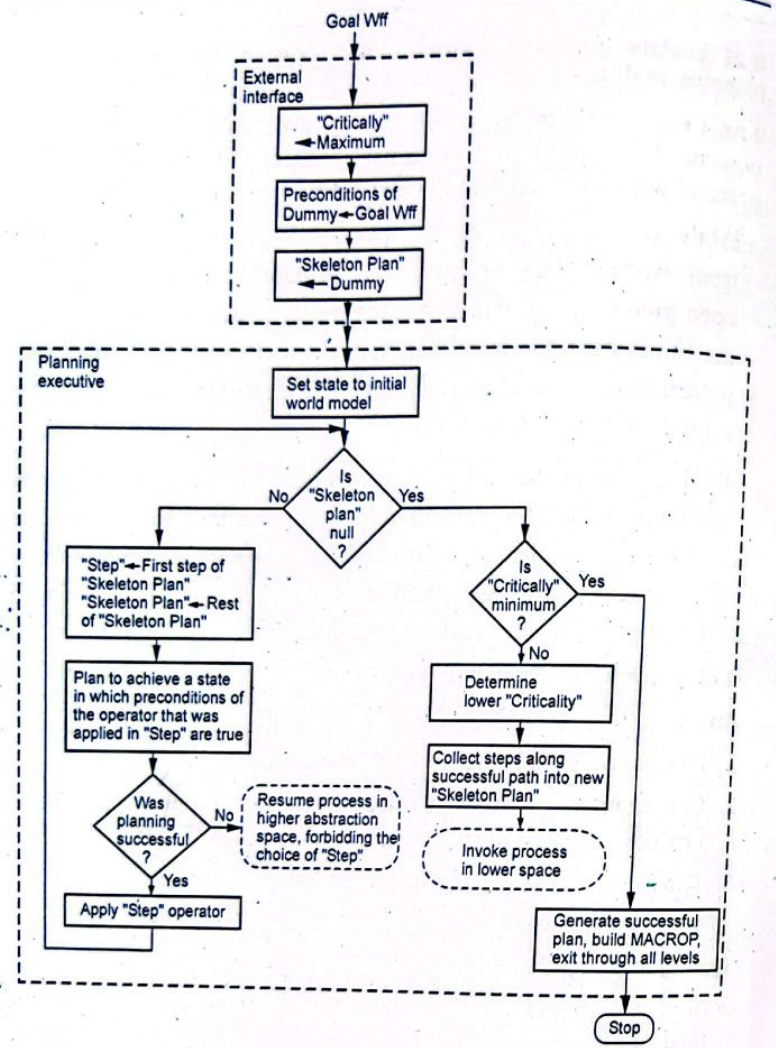


Fig. Q.22.1 : Planner

- Reach railway station(3)
- Pay-driver(1)
- Check in(1)

- Boarding train(2)
- Reach Rajasthan (3)

1st level plan

Buy Ticket (3), Reach Railway Station(3), Reach Rajasthan (3)

2nd level plan

Finding train (2), Buy ticket (3), Get taxi(2), Reach railway station (3), Boarding train(2), Reach Rajasthan (3).

3rd level plan (final)

Opening yatra.com (1), Finding train (2), Buy ticket (3), Get taxi(2), Reach Railway station (3), Pay-driver(1), Check in(1), Boarding train(2), Reach Rajasthan (3).

Q.23 Discuss various planning languages.

OR Compare STRIPS and ADL.

Ans. : • Language should be expressive enough to explain a wide variety of problems and restrictive enough to allow efficient algorithms to operate on it.

• Planning languages are known as action languages.

Stanford Research Institute Problem Solver (STRIPS)

• Richard Fikes and Nils Nilsson developed an automated planner called STRIPS (Stanford Research Institute Problem Solver) in 1971.

• Later on this name was given to a formal planning language. STRIPS is foundation for most of the languages in order to express automated planning problem instances in current use.

Action Description Language (ADL)

ADL is an advancement of STRIPS. Pednault proposed ADL in 1987.

Comparison between STRIPS and ADL

Sr. No.	STRIPS language	ADL
1.	Only allows positive literals in the states, **For example :** A valid sentence in STRIPS is expressed as ⇒ Intelligent^ Beautiful.	Can support both positive and negative literals. **For example :** Same sentence is expressed as ⇒ ¬ Stupid ¬ Ugly
2.	Makes use of closed - world assumption (i.e. Unmentioned literals are false)	Makes use of Open World Assumption (i.e. unmentioned literals are unknown)
3.	We only can find ground literals in goals. **For example :** Intelligent ∧ Beautiful.	We can find quantified variables in goals. **For example :** ∃x At (P1, x) ∧ At(P2, x) is the goal of having P1 and P2 in the same place in the example of the blocks
4.	Goals are conjunctions **For example :** (Intelligent ∧ Beautiful).	Goals may involve conjunctions and disjunctions **For example :** (Intelligent ∧ (Beautiful ∨ Rich)).
5.	Effects are conjunctions	Conditional effects are allowed : When P : E means E is an effect only if P is satisfied
6.	Does not support equality.	Equality predicate (x = y) is built in.
7.	Does not have support for types	Supported for types **For example :** The variable p : Person

Q.24 Explain planning for block world puzzle problem.

Ans. : Standard sequence of actions is

1. Grab Z and Pickup Z
2. Then place Z on the table
3. Grab Y and Pickup Y
4. Then stack Y on Z
5. Grab X and Pickup X
6. Stack X on Y

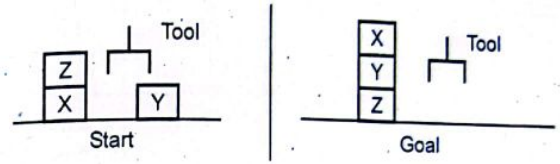


Fig. Q.24.1

- Elementary problem is that framing problem in AI is concerned with the question of what piece of knowledge or information is pertinent to the situation.
- To solve this problem we have tom make an Elementary Assumption which is a Closed world assumption. (i.e. If something is not asserted in the knowledge base then it is assumed to be false, this is also called as "Negation by failure")
- Standard sequence of actions can be given as for the block world problem :

on(Y, table)	on(Z, table)
on(X, table)	on(Y, Z)
on(Z, X)	on(X, Y)
hand empty	hand empty
clear(Z)	clear(X)
clear(Y)	

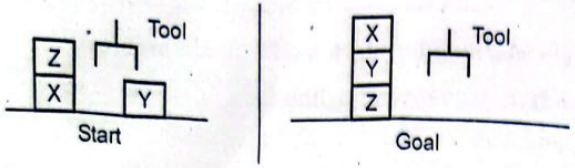


Fig. Q.24.2

- We can write four main rules for the block world problem as follows :

	Rule	Precondition and deletion list	Add list
Rule 1	pickup(X)	hand empty, on(X,table), sclear(X)	holding(X)
Rule 2	putdown(X)	holding(X)	hand empty, on(X,table), clear(X)
Rule 3	stack(X,Y)	holding(X), clear(Y)	on(X,Y), clear(X)
Rule 4	unstack(X,Y)	on(X,Y), clear(X)	holding(X), clear(Y)

- Based on the above rules, plan for the block world problem : Start → goal can be specified as follows :

1. unstack(Z,X) 2. putdown(Z) 3. pickup(Y)
4. stack(Y,Z) 5. pickup(X) 6. stack(X,Y)

- Execution of this plan can be done by making use of a data structure called "Triangular Table".
- In a triangular table there are N + 1 rows and columns. It can be seen from the Fig. Q.24.4 that rows have 1 → n + 1 condition and for columns 0 → n condition is followed. The first column of the triangular table indicates the starting state and the last row of the triangular table indicates the goal state.

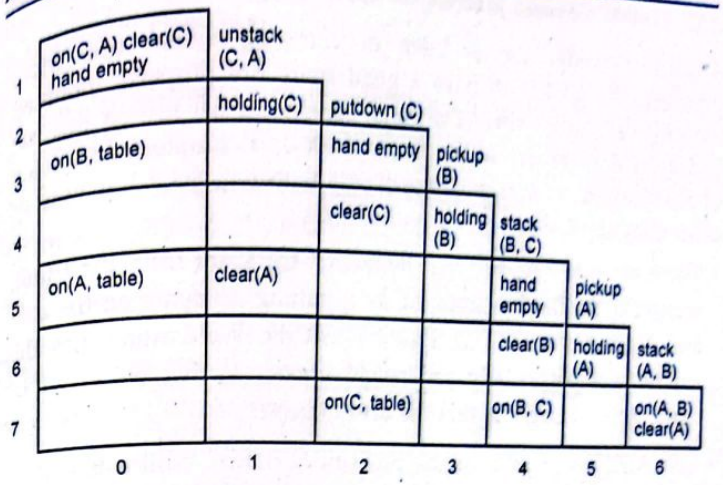


Fig. Q.24.3

- With the help of triangular table a tree is formed as shown below to achieve the goal state :

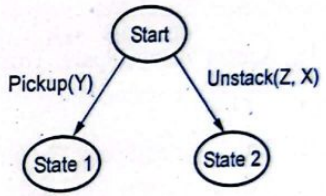


Fig. Q.24.4

- An agent (in this case robotic arm) can have some amount of fault tolerance. Fig. Q.24.5 shows one such example.

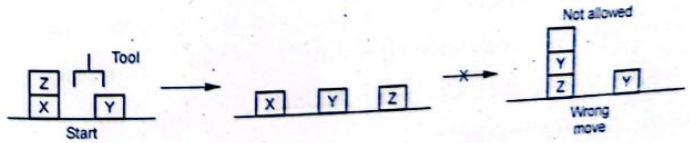


Fig. Q.24.5

Q.25 Explain planning problem for spare tire problem.

Ans. : • Consider the problem of changing a flat tire. More precisely, the goal is to have a good spare tire properly mounted onto the car's axle, where the initial state has a flat tire on the axle and a good spare tire in the trunk. To keep it simple, our version of the problem is a very abstract one, with no sticky lug nuts or other complications.

- There are just four actions : Removing the spare from the trunk, removing the flat tire from the axle, putting the spare on the axle and leaving the car unattended overnight. We assume that the car is in a particularly bad neighborhood, so that the effect of leaving it overnight is that the tires disappear.
- The ADL description of the problem is shown. Notice that it is purely propositional. It goes beyond STRIPS in that it uses a negated precondition, ¬ At(Flat, Axle), for the PutOn(Spare, Axle) action. This could be avoided by using Clear (Axle) instead.

Solution using STRIPS

- Init(At(Flat, Axle) ∧ At(Spare, Trunk))
- Goal(At(Spare, Axle)) Action(Remove(Spare, Trunk),
- PRECOND : At(Spare, Trunk)
- EFFECT : ¬ At(Spare, Trunk) ∧ At(Spare, Ground))
- Action(Remove(Flat, Axle),
- PRECOND : At(Flat, Axle)
- EFFECT : ¬ At(Flat, Axle) ∧ At(Flat, Ground))
- Action(PutOn(Spare, Axle),
- PRECOND : At(Spare, Ground) ∧ ¬ At(Flat, Axle)
- EFFECT : ¬ At(Spare, Ground) ∧ At(Spare, Axle))
- Action(LeaveOvernight)
- PRECOND

EFFECT : ¬ At(Spare, Ground) ∧ ¬ At(Spare, Axle) ∧ ¬ At(Spare, Trunk) ∧ ¬ At(Flat, Ground) ∧ ¬ At(Flat, Axle))

6.2 : Hierarchical Planning, Planning and Acting in Nondeterministic Domains, Time, Schedules, and Resources, Analysis of Planning Approaches, Limits of AI, Ethics of AI, Future of AI, AI Components, AI Architectures

Q.26 Explain how blocks world planning is carried out by agent in continuous environment.

Ans. : Example : Blocks world

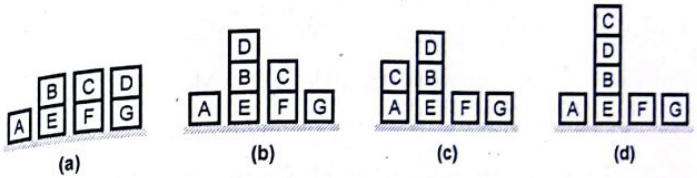


Fig. Q.26.1 The sequence of states as the continuous planning agent tries to reach the goal state on (C, D) ∧ On (D, B), as shown in (d)

- Assume a fully observable environment.
- Assume partially ordered plan.
- The start state is (a). At (b), another agent has interfered, putting D on B.
- At (C), the agent has executed Move (C, D) but has failed, dropping C on A instead.
- It retries Move (C, D), reaching the goal state (d).
- Initial state (a)
- Action (Move(x, y),

PRECOND : Clear (x) ∋ Clear(y) ∋ On(x, z).

EFFECT : On(x, y) ∋ Clear(z) ∌ On(x, z) ∌ Clear(y) .)

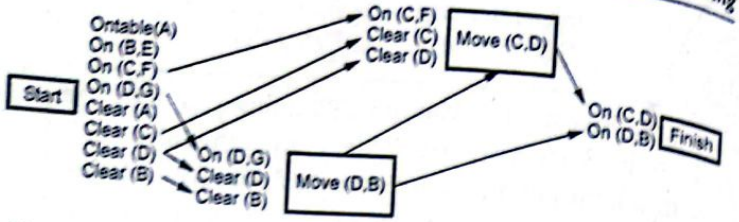


Fig. Q.26.2 The initial plan constructed by the continuous planning agent. The plan is indistinguishable, so far, from that produced by a normal partial-order planner

- The agent first need to formulate a goal : On(C, D) ∋ On(D, B).
- Plan is created incrementally, return NoOp and check percepts.
- Assume that percepts don't change and this plan is constructed.
- Ordering constraint between Move(D, B) and Move(C, D).
- Start is label of current state during planning.
- Before the agent can execute the plan, nature intervenes : D is moved onto B.

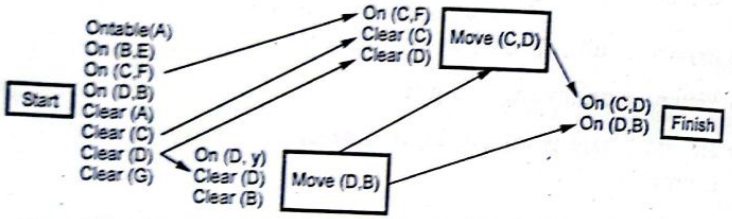


Fig. Q.26.3 After someone else moves D onto B, the unsupported links supplying clear (B) and on (D, G) are dropped, producing this

- Start contains now On(D, B).
- Agent perceives : Clear(B) and On(D, G) are no longer true.
- Update model of current state (Start).
- Causal links from Start to Move(D, B) (Clear(B) and On(D, G)) no longer valid.

- Remove causal relations and two PRECOND of Move(D, B) are open.
- Replace action and causal links to Finish by connecting Start to Finish.

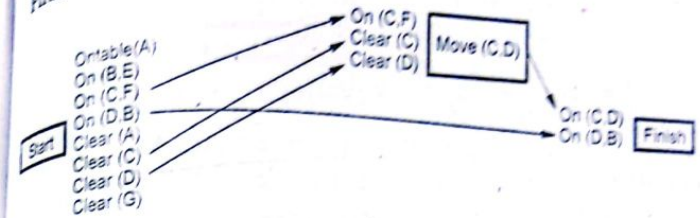


Fig. Q.26.4 The link supplied by Move (D, B) has been replaced by one from start and the now-redundant step Move (D, B) has been

- Extending : Whenever a causal link can be supplied by a previous step.
- All redundant steps (Move(D, B) and its causal links) are removed from the plan.

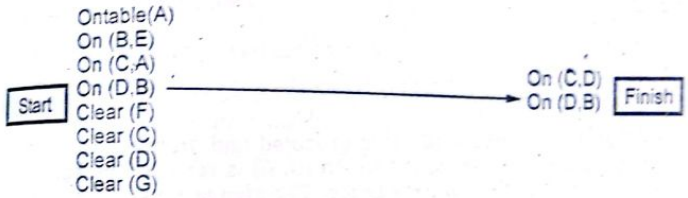


Fig. Q.26.5 After Move (C, D) is executed and removed from the plan, the effects of the start step reflect the fact that C ended upon A instead of the intended D. The goal precondition On (C, D) is still open

- Execute new plan, perform action Move(C, D).
- This removes the step from the plan.
- Execute new plan, perform action Move(C, D).
- Assume agent is clumsy and drops C on A.
- No plan but still an open PRECOND.

- Determine new plan for open condition.
- Again Move(C, D).
- The open precondition is resolved by adding Move(C, D) back in.

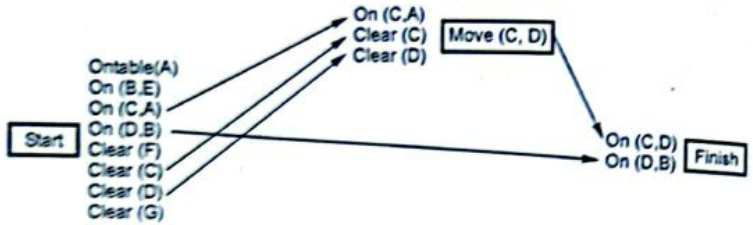


Fig. Q.26.6 The open condition is resolved by adding move (C, D) back in. Notice the new bindings for the preconditions

Fig. Q.26.7 After moves (C, D) is executed and dropped from the plan, the remaining open condition On (C, D) is resolved by adding a causal link from the new start step. The plan is now completed

- There are new bindings for the preconditions.
- Similar to POP.
- On each interation find plan-flaw and fix it.
- Possible flaws : Missing goal, open precondition, causal conflict, unsupported link, redundant action, unexpected action, unnecessary historical goal.

Q.27 Discuss time, schedule and resource with reference to planning of AI agent.

Ans. : Time, Schedule and Resources :

- In case of classical planning, the representations are about what is to done and in what order, but they cannot represent time. i.e. classical planner can not specify how long an action takes and when it occurs. For example, the Airline schedule planners can produce a schedule for an airline that says which planes are assigned to which flights, but no information about departure and arrival times is specified.
- While planning for real wold problems, only ordering of sub problem doesn't solve the problem completely. There might be time or schedule based constraints associated with the problem. For example, an airline staff who are on one flight cannot be on another at the same time. While planning we have to consider all the time, schedule and resource constraints.
- One of the approach is "plan first, schedule later": That is, first we consider the ordering constraints in the planning phase and then add temporal information to satisfy the resource and deadline conditions.
- Let's consider a typical job-shop scheduling problem. It consists of a set of jobs, each of which consists a collection of actions with ordering constraints among them. Each action has a duration and a set of resource constraints required by the action. Resource constraint details about the type of resource (e.g., bolts, wrenches, or pilots), the amount of that resource required (e.g. 1, 2, 1) and whether that resource is consumable (e.g., the bolts are consumed) or reusable (e.g., a staff is occupied during a flight but is available again when the flight is over).
- Resources can be made available or produced by actions by freeing them or as output of an action, including manufacturing, growing and resupply actions.
- A solution to a job-shop scheduling problem must specify the start times for each action and must satisfy all the tern pond ordering constraints and resource constraints.

- To evaluate the solution we consider the total time taken by the plan, which is called as "makespan".

```
Jobs (AddEngine1 AddWheels1 Inspect1)
        (AddEngine2 AddWheels2 Inspect2))
Resources (EngineHoists (1), WheelStations (1), Inpsectors (2), Lug
Nuts (500))
Actions (AddEngine1, DURATION:30,
        USE:EngineHoists(1))
Action(AddEngine 2, DURATION:60,
        USE:EngineHoists(1))
Action(AddWheels1, DURATION:30,
        CONSUME:LugNuts (20), USE :WheelStations(1))
Action(AddWheels2, DURATION:15,
        CONSUME:LugNuts (20), USE :WheelStations(1))
Action(Inspect1, DURATION:10,
        USE:Inspectors(1))
```

Fig. Q.27.1 : A job-shop scheduling problem for assembling two cars, with resource constraints. The notation A B means that action A must precede action B

- Shown in the example, in Fig. Q.27.1, a problem involving the assembly of two calls. The problem consists of two jobs, each of the form [AddEngine, AddWheels, Inspect].
- In general, resources statement list the available resources and gives the number of each type available in the beginning.
- While specifying the action, we give details of the duration, resource needed for that action and consumables of the action.
- If the action requires any resources to be used while performing the action, they specified by USE element. These resources will be freed at the end of the action.

Q.28 Explain hierarchical planning in detail.

Ans. : • Hierarchical planning is also called as **plan decomposition**. Generally plans are organized in a hierarchical format.

- Complex actions can be decomposed into more primitive actions and it can be denoted with the help of links between various states at different levels of the hierarchy. This is called as operator expansion.
- **For example**

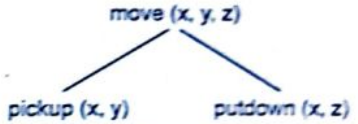


Fig. Q.28.1 Operator expansion

- Fig. Q.28.2 shows, how to create a hierarchical plan to travel from some source to a destination. Also you can observe, at every level how we follow some sequence of actions.

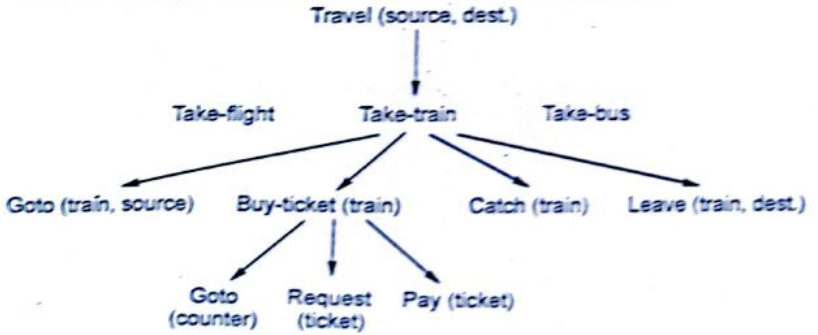


Fig. Q.28.2 Hierarchical planning example

Q.29 Discuss hierarchy of actions in planning.

Ans. : • In terms of major and minor actions, hierarchy of actions can be decided. Minor activities would cover more precise activities to accomplish the major activities. In case of above example, we can have railway Ticket Booking, Hotel Booking, Reaching Rajasthan, Staying and enjoying there, coming back are the major activities.

- While take a taxi to reach railway station, Have candle light dinner in palace, Take photos, etc are the Minor activities.
- In real world there can be complex problems.
- **For example :** A captain of a cricket team plans the order of 4 bowlers in 2 days of a test match (180 overs). Number of probabilities : $4^{180} = 16^{90}$.
- Motivation behind this planning is to reduce the size of search space. For plan ordering we have to try out a large number of possible plans. With plan hierarchies we have limited ways in which we can select and order primitive operators.
- In hierarchical planning major steps are given more importance. Once the major steps are decided we attempt to solve the minor detailed actions.
- It is possible that major steps of plan may run into difficulties at a minor step of plan. In such case we need to return to the major step again to produce appropriately ordered sequence to devise the plan.

Q.30 Explain various types of planning method for handling indeterminacy.

Ans. : • In earlier sections, we have discussed planning that is observable and deterministic. Real world problems however are not predictable and completely unobservable. The planning and acting on real world problems require more sophisticated approach.

- One of the most important characteristic of real world is uncertainty. In an uncertain environment, it is very important for agent to rely upon its percepts (series of past experience).
- Whenever some unexpected condition encountered, agent should refer its percept and should change course of action accordingly. In other words agent should be able to cancel or replace the currently executing plan with some other more suitable and reliable plan if something unexpected happens.

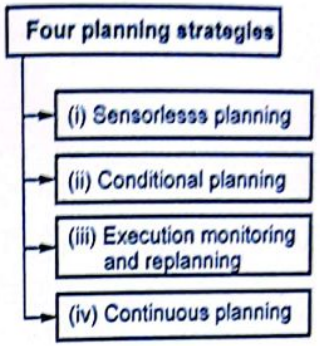


Fig. Q.30.1 Planning strategies

- It should be noted that real world itself is not uncertain but human perception related to world is uncertain. In artificial intelligence, we try to give human perception ability to machine, and hence machine also receives the perception of uncertainty about the real world. So machine has to deal with incomplete and incorrect information like human does.
- Determining the condition of state depends on available knowledge. In real world, knowledge availability is always limited, so most of the time, conditions are non deterministic.
- The amount or degree of indeterminacy depends upon the knowledge available. The inter determinacy is called "bounded indeterminacy" when actions can have unpredictable effects.
- Four planning strategies are there for handling indeterminacy :

(i) Sensorless planning : Sensorless planning is also known as conformant planning. These kinds of planning are not based on any perception. The algorithm ensures that plan should reach its goal at any cost.

(ii) Conditional planning : Conditional plannings are sometimes termed as contingency planning and deals with bounded indeterminacy discussed earlier. Agent makes a plan, evaluate the plan and then execute it fully or partly depending on the condition.

(iii) Execution monitoring and replanning : In this kind of planning agent can employ any of the strategy of planning discussed earlier. Additionally it observes the plan execution and if needed, replan it and again executes and observes.

(iv) Continuous planning : Continuous planning does not stop after performing action. It persist over time and keeps on planning on some predefined events.

These events include any type unexpected circumstance in environment.

Q.31 Write a note on multi-agent planning.

Ans. : • Whatever planning we have discussed so far, belongs to single user environment. Agent acts alone in a single user environment.

- When the environment consists of multiple agent to, then the way a single agent plan its action get changed.
- We have a glimpse of environment where multiple agents have to take actions based on current state. The environment could be co-operative or competitive. In both the cases agent's action influences each other.
- Few of the multi agent planning strategies are listed below :

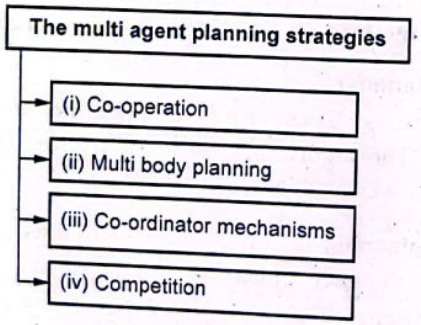


Fig. Q.31.1 Multi-agent planning strategies

(i) Co-operation : In co-operation strategy agents have joint goals and plans. Goals can be divided into subgoals but ultimately combined to achieve ultimate goal.

(ii) Multibody planning : Multi body planning is the strategy of implementing correct joint plan.

(iii) Co-ordination mechanisms : These strategies specify the co-ordination between co-operating agents. Co-ordination mechanism is used in several co-operating plannings.

(iv) Competition : Competition strategies are used when agents are not co-operating but competing with each other. Every agent wants to achieve the goal first.

Q.32 What is conditional planning ? Explain with suitable example.

Ans. : • Conditional planning has to work regardless of the outcome of an action.

- Conditional planning can take place in Fully Observable Environments (FOE) where the current state of the agent is known environment is fully observable. The outcome of actions cannot be determined so the environment is said to be nondeterministic.
- In conditional planning we can check what is happening in the environment at predetermined points of the plan to deal with ambiguous actions.
- It can be observed from vacuum world example, conditional planning needs to take some actions at every state and must be able to handle every outcome for the action it takes. A state node is represented with a square and chance node is represented with a circles.
- For a state node we have an option of choosing some actions. For a chance node agent has to handle every outcome.

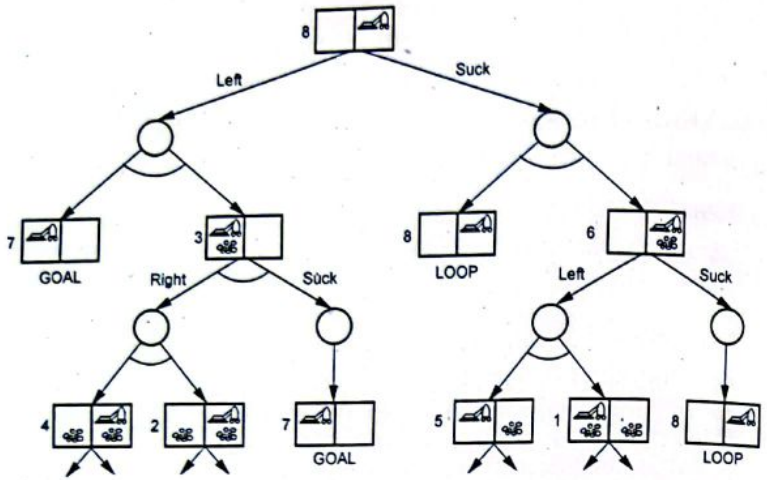


Fig. Q.32.1 Conditional planning - vacuum world example

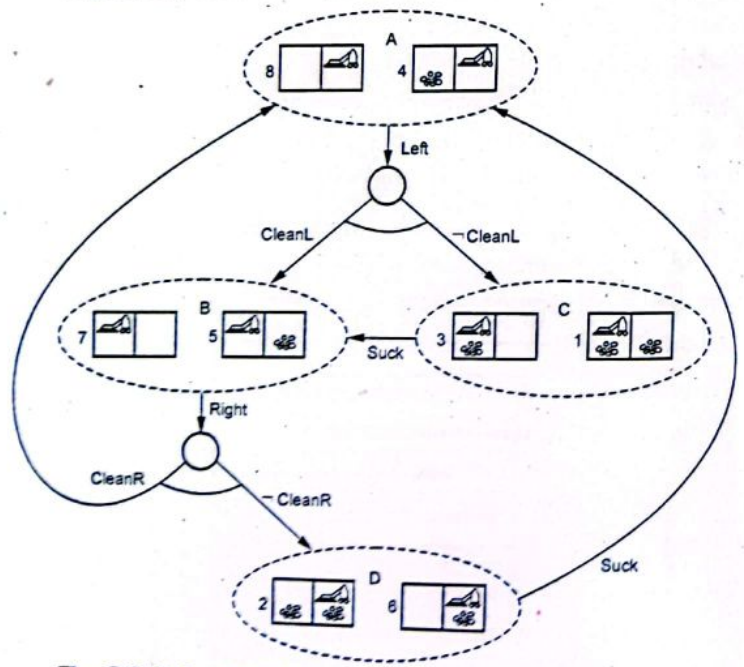


Fig. Q.32.2 Conditional planning - vacuum world example (condition 2)

- Conditional planning can also take place in the Partially Observable Environments (POE) where, we cannot keep a track on every state. Actions can be uncertain because of the imperfect sensors.
- In vacuum agent example if the dirt is at right and agent knows about right, but not about left. Then, in such cases dirt might be left behind when the agent, leaves a clean square. Initial state is also called as a state set or a belief state.
- Sensors play important role in conditional planning for partially observable environments. Automatic sensing can be useful; with automatic sensing an agent gets all the available percepts at every step. Another method is active sensing, with which percepts are obtained only by executing specific sensory actions.

Q.33 Discuss planning with operators.

Ans. : • In typical planning problems, we have set of preconditions which must be satisfied before we start with actual actions and there are effects of the action. These pre conditions and effects can be represented as operators.

- For real world planning problems these operators represent some complex control problem on their own. Take example of block world problem, where a robotic arm is used to pick and place the blocks. In this case number of contingencies can occur like, motor failure or obstruction by an obstacle.
- This will potentially affect the outcome and applicability of the action. Hence to incorporate all the possibilities of the real world scenarios operators must be complex in nature.
- However, the complex operators will make the plan untraceable, which the planner will never be interested in. So there arises a conflict. By simplifying the operators, the reliability itself will be challenged and by using complex operators, we may not be able to solve the planning problem.
- We have to maintain both simplicity and reliability in the planning operators. In order to solve this matter, if we can identify those details of the world dynamics likely to become relevant within our distribution of problems and structure our operators accordingly, we may achieve the goal to certain extent.

- Say in case of block world problem, possibility of the arm getting stuck by an obstacle is high only when it is placed in crowded environment; which is mostly not likely. So while designing the planner, we may skip this detail.
- Fig. Q.33.1 depicts the scenario of classical planning proceeds.

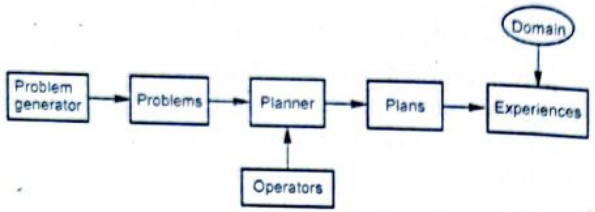


Fig. Q.33.1 Classical planning framework

- The **problem generator** produces planning problems according to some fixed but unknown probabilities. The **planner** takes in the problems, along with a library of operators and outputs plans, which, when applied to the domain, result in some set of experiences. In this framework, the operators are designed by a domain expert and are immutable.
- As the domain expert designs the operators, they are kept simple in order to keep the planning phase manageable. Hence there is less specificity of the operators. Many unimportant details might get covered while losing some critical specifications.
- For example, operators like motor failure in case of robot are is introduced which is very less likely as all the robot are well maintained. While, an important aspect such as the presence of obstacles in the arm's path might get ignored; which is very critical if our tasks frequently take place in a crowded environment.
- In order to avoid such situations, the planning process is supplemented by a new operator design module, detailed in Fig. Q.33.2.
- Instead of planning with an a priori fixed set of operators, specified by a domain expert, this module takes advantage of the observed world experiences to tailor operator definitions to the particular distribution of planning problems.

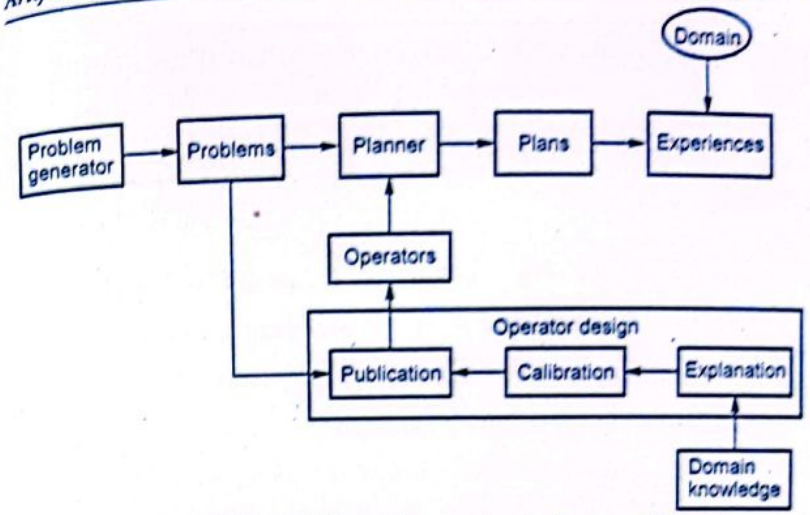


Fig. Q.33.2 Addition of operator design module

- The domain expert specifies only a general body of domain knowledge. The knowledge can be of varying specificities and need not be consistent.
- Three submodules make up the operator design module.
  1. **Explanation submodule :** It is used to associate observed world dynamics with a consistent causal model, calling on explanations from the knowledge base.
  2. **Calibration mechanism :** It associates precise numerical functions to the qualitative causal structure.
  3. **Publication module :** Assesses the capabilities of our operators against the distribution of planning problems and produces a minimal sufficient set of operator definitions for use by the planner.

Example :
In case of block world problem, we can have operator as below,
Op(ACTION : Unstack(b, x),
PRECOND : On(b, x) and Clear(b),
EFFECT : On(b, Table) and Clear(x) and !On(b, x))
Knowledge Representation :

END...

SOLVED MODEL QUESTION PAPER (End Sem)
Artificial Intelligence

T.E. (AI&DS) Semester - V [As Per 2019 Pattern]
T.E. (Computer) Semester - VI [As Per 2019 Pattern]

Time : 2 $\frac{1}{2}$ Hours] **[Maximum Marks : 70**

N.B. : i) Attempt Q.1 or Q.2, Q.3 or Q.4, Q.5 or Q.6, Q.7 or Q.8.

ii) Neat diagrams must be drawn wherever necessary.

iii) Figures to the right side indicate full marks.

iv) Assume suitable data, if necessary.

Q.1 a) *Explain Min-Max search procedure with an example.* **(Refer Q.4 of Chapter - 3)** **[9]**

b) *Explain alpha-beta cut off search with an example. State a case when to do alpha pruning.* **(Refer Q.6 of Chapter - 3)** **[9]**

OR

Q.2 a) *Explain α - β pruning procedure. Mark the nodes in the Fig. 1 which will prune out ?* **(Refer Q.8 of Chapter - 3)** **[9]**

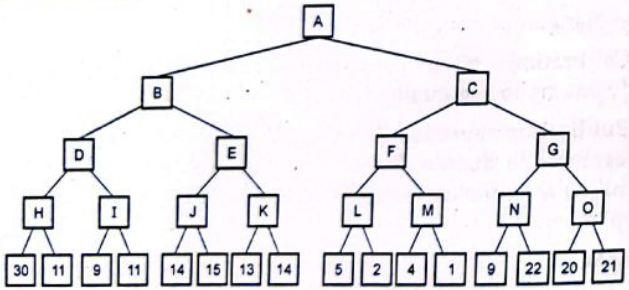


Fig. 1

b) *Solve the following crypt arithmetic problem.*

```
  DONALD
+ GERALD
  ROBERT
```

(Refer Q.14 of Chapter - 3) **[5]**

c) *Discuss AO* algorithm. Give one example where AO* is suitable to apply.* **(Refer Q.16 of Chapter - 3)** **[4]**

Q.3 a) *Represent following facts using propositional logic.*
i) All pompeian are Romans. ii) Caesar was a ruler.
iii) All Romans were either loyal to Caesar or hated him.
iv) Everyone is loyal to someone.
v) People only try to assassinate rulers they are no loyal.
vi) Marcus tried to assassinate ceasar. **(Refer Q.2 of Chapter - 4)** **[9]**

b) *What is propositional logic ? Explain with example.* **(Refer Q.3 of Chapter - 4)** **[8]**

OR

Q.4 a) *Explain how to have inference using propositional logic. Also explain resolution using propositional logic.* **(Refer Q.8 of Chapter - 4)** **[8]**

b) *What are higher-order logic ?* **(Refer Q.15 of Chapter - 4)** **[9]**

Q.5 a) *Represent Wumpus world using propositional logic along inference process.* **(Refer Q.7 of Chapter - 4)** **[9]**

b) *Consider the facts :*
1. Anyone whom Mary loves is a football star.
2. Any student who does not pass does not play.
3. John is a student.
4. Any student who does not study does not pass.
5. Anyone who does not play is not a football star.
Prove using resolution "If John does not study, then, Mary does not love John". **(Refer Q.2 of Chapter - 5)** **[9]**

OR

Q.6 a) *Explain the steps of unification in predicate logic. Also discuss the steps of converting predicate logic wffs to clause form.* **(Refer Q.6 of Chapter - 5)** **[8]**

b) *What is universal quantifier ?* **(Refer Q.16 of Chapter - 4)** **[7]**

c) *Determine whether the following argument is valid : "If a baby is hungry, then baby cries. If the baby is not mad, then he does not cry. If a baby is mad, then his face looks abnormal. Therefore, if a baby is hungry, then his face looks abnormal."* **(Refer Q.11 of Chapter - 5)** **[3]**

Q.7 a) *Consider the facts :*
1) Every child loves santa 2) Every child loves every candy.
3) Anyone who loves some candy is not a nutrition fanatic.
4) Anyone who eats any pumpkin is a nutrition fanatic.
5) Anyone who buys any pumpkin either carves it or eats it.
6) John buys a pumpkin.
7) Lifesavers is a candy.
Use resolution and prove : If John is a child, then John carves some pumpkin. **(Refer Q.5 of Chapter - 5)** **[9]**

b) *Jacks owns a dog.*
Every dog owner is an animal lover.
No animal lover kills an animal.
Either Jack or Curiosity killed the cat, who is named Tuna. By using Resolution prove that.
"Did curiosity kill the cat".
(Refer Q.14 of Chapter - 5) **[8]**

OR

Q.8 a) *Solve the blocks world problem using forward state-space search and backward state-space search.* **(Refer Q.3 of Chapter - 6)** **[9]**

b) *Explain planning problem.* **(Refer Q.12 of Chapter - 6)** **[8]**

END...

JUNE - 2022 [5870]-1135 (COMP.) Solved Paper

Course 2019

Time : 2 $\frac{1}{2}$ Hours] **[Maximum Marks : 70**

Instructions to the candidates :

N.B. : 1) Answer Q.1 or Q.2, Q.3 or Q.4, Q.5 or Q.6, Q.7 or Q.8.

2) Neat diagrams must be drawn wherever necessary.

3) Assume suitable data, if necessary.

Q.1 a) *Explain Alpha - Beta Tree search and cutoff procedure in detail with example.* **(Refer Q.6 of Chapter - 3)** **[9]**

b) *What are the issues that need to be addressed for solving esp efficiently ? Explain the solutions to them.* **[9]**

Ans. : • **While solving CSP one has to start from an empty solution and set the variables one by one until all the values are assigned.** When setting a variable, one should consider only the values consistent with those of the previously set variables. If it is realized that the variable can not be set without violating a constraint, then it is required to backtrack to one of those values set before. Then, the variable is set to next value in its domain and continue with other unassigned variables.

- **Although the order in which the variables are assigned to their values don't affect the correctness of the solution, it does influence the algorithm's efficacy.** The same goes for the order in which the values in a variable's domain are chosen. Moreover, it turns out that whenever a variable is set, one can infer which values of the other variables are inconsistent with the current assignment and discard them without traversing the whole sub-tree.
- **The implementation of variable selection, value ordering and inference are the issues that are required to be handled so as to solve CSP efficiently.** In traditional search, the algorithms performance is improved by formulating problem - specific heuristics that guided the

search efficiently. However, it turns out that there are domain - independent techniques can be used to speed up the backtracking solver for CSPs. Below each issue is discussed in detail.

1. **Inference** - Is a step taken right after setting a variable X to a value 'v' in its domain. Then one **iterates over all the variables connected to X in the constraint (hyper)graph and remove the values inconsistent with the assignment X = v from their domains.** That way, one can prune the **search tree** as one can discard impossible assignments in advance. **This technique is known as forwarding checking.** Further, whenever one prunes a value v from D_i, one can check what happens to the neighbors of X_i in the graph. More specifically, for every X_j that is connected to X_i via a constraint $C_k \neq C$, it can be checks if there's a value $w \neq D_j$ such that $X_j = w$ satisfies C_k only if $X_i = v$. Any such value w can be discarded because any assignment containing $X_j = w$ is impossible too. However, this assumes that all the constraints are binary.
2. **Variable selection** - A straightforward strategy for variable selection is to consider the variables in a static order that is fixed before solving the CSP at hand. Or, one may select the next variable randomly. However, those aren't efficient strategies. **If there's a variable with only one value in its domain, it makes sense to set it before others.** The reason is that it optimizes the worst-case performance. In the worst-case scenario, one can't complete the current partial assignment, so the backtracking search will return failure after traversing the whole sub-tree under that assignment. But, the variable with only one legal value is the most likely to conflict with other variables at shallower levels of the sub-tree and reveal its inconsistency. In general, this is the rationale for the Minimum-Remaining-Values heuristic (MRV). **MRV chooses the variable with the fewest legal values remaining in its domain.**
3. **Value ordering** - The value ordering heuristic follows the opposite logic. When setting X to a value, one should choose the value that rules out the fewest values from the domains of the X's neighbors. The intuition behind this choice is that one needs only one solution, and it's more likely for a larger sub-tree to contain it than for a smaller one. This strategy is called as the Least-Constraining - Value

heuristic (LCV). In a way, LCV balances off the tendency of MRV to prune the search tree as much as possible. CSP solvers that incorporate both heuristics are usually very efficient in practice. However, it should be noted that MRV doesn't require the CSP to be binary, whereas LCV does. That's not a problem because one can convert any CSP to a binary form. However, if one uses custom-made inference rules that operate on higher-order constraints such as ALL_DIFFERENT, one should make sure to maintain both representations of the CSP during the execution of the backtracking solver.

OR

Q.2 a) *Explain in detail the concepts of back tracking and constraint propagation and solve the N-queen problem using these algorithms.* **[9]**

Ans. : If we apply BFS to generic CSP problems then we can notice that the branching factor at the top level is 'nd', because any of 'd' values can be assigned to any of 'n' variables. At the next level, the branching factor is '(n – 1) d' and so on for n levels. Therfore a tree with 'n! * d^n' leaves is generated even though there are only 'd^n' possbile complete assignments.

Basic steps in backtracking search :

1. Depth-first search selects values for single variable at a given time.
2. Apply constraint to the variable when variable is selected.
3. Backtracking action-performed if a variable has no legal values left to assign.
4. Backtracking search is basic uninformed algorithm for CSPs.

Backtracking search algorithm :

1. Consider a CSP problem.
2. Apply backtracking search. If backtracking search successful returns a solution else, a failure state which return a procedure recursive-backtracking.
3. Procedure recursive-backtracking starts with empty set and takes input as CSP problem.

If complete assignement possible or assignment done then return actual assignment.

4. Variable is assigned a specific value.
5. The relative constraint is a set which is taken as input.
6. If value is complete and consistent according to constraints then assign value to variable, add that to list.
7. Call the recursive backtracking until result or failure is reached.
8. Every time recursive-backtracking sustains result (i.e. assignment.)
9. If result is failure then remove variable with specific value from assignment. It will return failure status.

Constraint propagation :

- While selecting unassigned variable for computation if we look at some of the constraints earlier in the search (or even before search begins), we can drastically reduce search space. **Following are techniques which are helpful for earlier constraints checks.**

Forward Checking (FC) :

1. Forward checking is one of the potential technique which uses constraints more effectively during search.
2. It removes values in neighbouring unassigned variables domain that conflict with assigned variable.
3. Forward checking uses MRV to select assigned variable.
4. Backtracking search is performed if failure occurs.
5. Search terminates when any variable has no legal values.
6. Generic method,
   - i) Assigns variable X.
   - ii) Forward checking remarks unassigned variable Y connected to X.
   - iii) Remove values from D, where value is inconsistent with X.

Constraint propagation process :

1. A combined approach of heuristic plus forward checking gives more reliable, accurate and efficient results than a singular approch.
2. The forward checking propagates information from assigned to unassigned variables but can not avoid or detect all failure.
3. Constraint propagation repeatedly enforces constraints locally.
4. The idea of arc consistency provides a fast method of constraint propagation that is substantially stronger than forward checking. Here "arc" refers to a directed arc in the constraint graph.

Constraint propagation using arc consistency :

1. It is fast method of constraint propagation.
2. $X \rightarrow Y$ is consistent if (for every value of X there is some allowed value Y. For example $[V_2 \rightarrow V_1$, is consistent iff V_1 = Red, V_2 = Blue] (i.e. for every value of x in X there is some allowed value y in Y). This is directed property example - $[V_1 = V_2]$

   $V_1 = V_2$ is consistent iff

   V_1 = Red and V_2 = Blue
3. As directed arcs between variables represent the domains of specified variables, they are consistent with each other.
4. Constraint propagation can be applied as preprocessing or propagation step.

   i) Before search-Preprocessing.

   ii) After search-Propagation.
5. The procedure for maintaining arc consistency, can be applied repeatedly.

Constraint propagation using K-consistency :

1. Arc consistency is not capable of detecing all inconsistencies. Partial assignments [V_1 = Red, V_2 = Red] are inconsistent.
2. K-consistency is very strong form of constraint propagation.

3. A CSP is K-consistency if for any set of K-1 variables and for any consistent assignment to those variable a consistent value can always be assigned to any K^{th} variable.

Note

1. Consistency means that each individual variable by itself is consistent; this is also called as node consistency.
2. Consistency is the same as arc consistency.
3. Consistency means that any pair of adjacent-variables can always be extended

   to a third neighboring variable; this is also called as path consistency.
4. Strongly, K-consistency graph exhibit some properties mentioned below -

   i) It is K-consistent.

   ii) It is also (K – 1) consistent, (K – 2) consistent all the way down to 1 consistent.
5. This is an idealist solution which requires O (nd) time instead of $O(n^2d^3)$.

- An exponential order time is required for establishing n-consistency in the worst case.

N-Queen Problem as CSP

- This problem, first posed in a German chess magazine almost 150 years ago, requires that 8 queens be placed on an 8 × 8 chessboard in such a way that no two queens attack each other, i.e. no two queens occupy the same row, column, or diagonal.
- This problem can be formally represented as a CSP in several different ways, and as we shall see, the choice of formal representation has a significant impact on the computational complexity of the problem.

- Suppose, for example, that we choose to represent the problem by numbering the queens 1 through 8 and then defining a set Z of 64 variables, one for each square of the chessboard, where each variable has domain D = {0, 1, 2, ... , 8} and a variable z having a value of i means that queen i occupies the corresponding square if $1 \leq i \leq 8$ or that the square is unoccupied if i = 0. This formulation of the problem yields a search space of cardinality $9^{64} \approx 1.18 \times 10^{61}$, making it impractical from a computational perspective.

- Consider instead a representation with a set Z of 8 variables $Q_1, Q_2, \ldots Q_8$ representing the 8 rows of the board, with the domain of each variable Q_i being the set $D_i = \{1, 2, \ldots, 8\}$; in this case the value of the variable Q_i represents the column in the i^{th} row in which a queen is placed. The constraints can then be specified as logical expressions of the form

$$i \neq j \Rightarrow Q_i \neq Q_j$$

(i.e., no two queens are in the same column) and

$$i \neq j \Rightarrow |Q_i - Q_j| \neq |i - j|$$

(i.e., no two queens are in the same diagonal). Note that we have implicitly incorporated one of the problem constraints (that no two queens occupy the same row) as well as our reasoning about the consequences of this constraint (it implies that each row must have precisely one queen in it) into our representation. Note too that the cardinality of the search space is now reduced to $8^8 \approx 1.68 \times 10^7$ states.

- This example clearly illustrates the need to carefully select a representation for a CSP from among the many formally valid possibilities, bearing in mind the computational complexity which each one entails.

- To find all solutions to the N-queens problem, the CP solver systematically tries all possible assignments of values to the variables, to see which ones give feasible solutions to the problem. In the 4-queens problem, the solver starts at the leftmost column and successively places one queen in each column, at a location that is not attacked by any previously placed queens.

Propagation and backtracking

There are two key elements to a CP search :

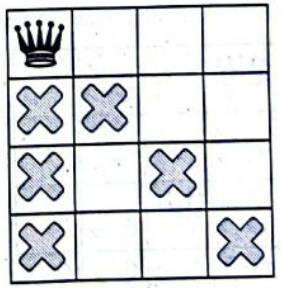
- Propagation - Each time the solver assigns a value to a variable, the constraints add restrictions on the possible values of the unassigned variables. These restrictions propagate to future variable assignments. For example, in the 4-queens problem, each time the solver places a queen, it can't place any other queens on the row and diagonals the current queen is on. Propagation can speed up the search significantly by reducing the set of variable values the solver must explore.

- *Backtracking* occurs when either the solver can't assign a value to the next variable, due to the constraints, or it finds a solution. In either case, the solver backtracks to a previous stage and changes the value of the variable at that stage to a value that hasn't already been tried. In the 4-queens example, this means moving a queen to a new square on the current column.

Solution using constraint satisfaction

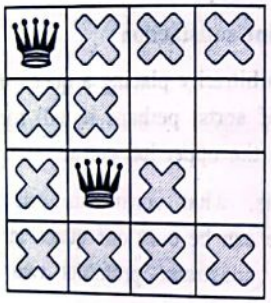
- The solver starts by arbitrarily placing a queen in the upper left corner. That's a hypothesis of sorts; perhaps it will turn out that no solution exists with a queen in the upper left corner.

- Given this hypothesis, what constraints can we propagate ? One constraint is that there can be only one queen in a column (the gray Xs below), and another constraint prohibits two queens on the same diagonal (the red Xs below).

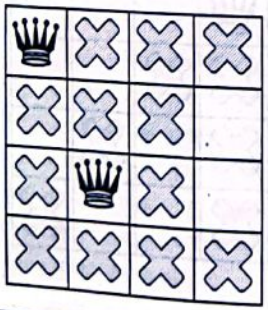


- Our third constraint prohibits queens on the same row :

- Our constraints propagated, we can test out another hypothesis, and place a second queen on one of the available remaining squares. Our solver might decide to place in it the first available square in the second column :

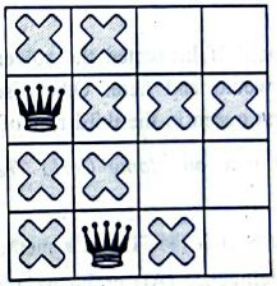


- After propagating the diagonal constraint, we can see that it leaves no available squares in either the third column or last row :

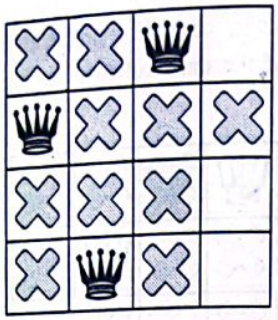


- With no solutions possible at this stage, we need to backtrack. One option is for the solver to choose the other available square in the second column. However, constraint propagation then forces a queen into the second row of the third column, leaving no valid spots for the fourth queen :

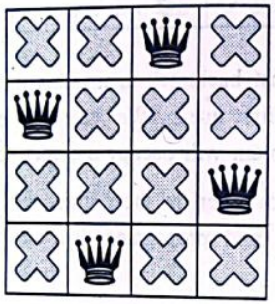
- And so the solver must backtrack again, this time all the way back to the placement of the first queen. We have now shown that no solution to the our queens problem will occupy a corner square.
- Since there can be no queen in the corner, the solver moves the first queen down by one, and propagates, leaving only one spot for the second queen :



- Propagating again reveals only one spot left for the third queen :



- And for the fourth and final queen :



- This is the first solution! If instructed the solver to stop after finding the first solution, it would end here. Otherwise, it would backtrack again and place the first queen in the third row of the first column.

b) *Write a short note on Monte Carlo Tree search and list its limitations.* **[5]**

Ans. : Monte Carlo Tree Search (MCTS) is a method for making optimal decisions in Artificial Intelligence (AI) problems, typically move planning in combinatorial games. It combines the generality of random simulation with the precision of tree search.

The basic MCTS algorithm is simple: a search tree is built, node by node, according to the outcomes of simulated playouts. The process can be broken down into the following steps.

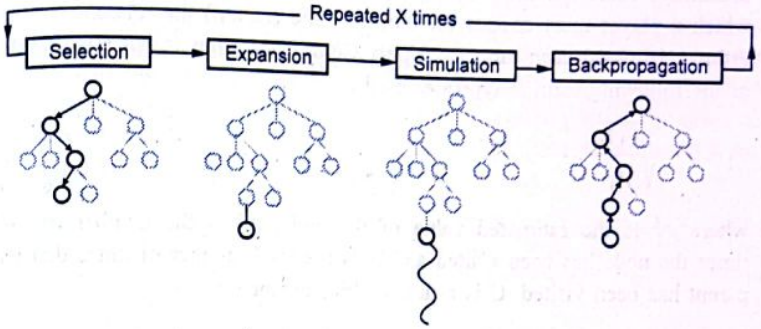


Fig. 1

1. **Selection :** Starting at root node R, recursively select optimal child nodes (explained below) until a leaf node L is reached.
2. **Expansion :** If L is a not a terminal node (i.e. it does not end the game) then create one or more child nodes and select one C.
3. **Simulation :** Run a simulated playout from C until a result is achieved.
4. **Backpropagation :** Update the current move sequence with the simulation result.

Each node must contain two important pieces of information : An estimated value based on simulation results and the number of times it has been visited.

In its simplest and most memory efficient implementation, MCTS will add one child node per iteration. Note, however, that it may be beneficial to add more than one child node per iteration depending on the application.

Node selection in MCTS

Bandits and UCB

Node selection during tree descent is achieved by choosing the node that maximises some quantity, analogous to the multiarmed bandit problem in which a player must choose the slot machine (bandit) that maximises the estimated reward each turn. An Upper Confidence Bounds (UCB) formula of the following form is typically used :

$$v_i + C \times \sqrt{\frac{\ln N}{n_i}}$$

where v_i is the estimated value of the node, n_i is the number of the times the node has been visited and N is the total number of times that its parent has been visited. C is a tunable bias parameter.

Exploitation vs Exploration

The UCB formula balances the exploitation of known rewards with the exploration of relatively unvisited nodes to encourage their exercise. Reward estimates are based on random simulations, so nodes must be visited a number of times before these estimates become reliable; MCTS estimates will typically be unreliable at the start of a search but converge to more reliable estimates given sufficient time and perfect estimates given infinite time.

Limitations of MCTS

1. As the tree growth becomes rapid after a few iterations, it requires a huge amount of memory.
2. There is a bit of a reliability issue with Monte Carlo Tree Search. In certain scenarios, there might be a single branch or path, that might lead to loss against the opposition when implemented for those turn-based games. This is mainly due to the vast amount of combinations and each of the nodes might not be visited enough number of times to understand its result or outcome in the long run. The MCTS algorithm, in its basic form, can fail to find reasonable moves for even games of medium complexity within a reasonable amount of time. This is mostly due to the sheer size of the combinatorial move space and the fact that key nodes may not be visited enough times to give reliable estimates.

MCTS algorithm needs a huge number of iterations to be able to effectively decide the most efficient path. MCTS search can take many iterations to converge to a good solution, which can be an issue for more general applications that are difficult to optimise. For example, the best Go implementations can require millions of playouts in conjunction with domain specific optimisations and enhancements to make expert moves, whereas the best GGP implementations may only make tens of (domain independent) playouts per second for more complex games. For reasonable move times, such GGPs may barely have time to visit each legal move and it is unlikely that significant search will occur.

c) *Apply constraint satisfaction method to solve following Problem*

SEND + MORE = MONEY. (TWO + TWO = FOUR, CROSS+ ROADS = DANGER) **[4]**

Ans. : i)

```
5  4   3   2   1
   S   E   N   D
+  M   O   R   E
   C3  C2  C1
-------------
  M O N E Y
```

where C_1, C_2, C_3 are the carry forward numbers upon addition.

From column 5, M = 1, since it is only carry-over possible from sum of 2 single digit number in column 4.

1. To produce a carry from column 4 to column 5 'S + M' is at least 9 so 'S = 8 or 9' so 'S+M = 9 or 10' and so 'O = 0 or 1' . But 'M = 1', so '0 = 0'.
2. If there is carry from column 3 to 4 then 'E = 9' & so 'N = 0'. But 'O = 0' so there is no carry and 'S = 9' and 'C_3 = 0'.
3. If there is no carry from column 2 to 3 then 'E = N' which is impossible, therefore there is carry and 'N = E+1' & 'C_2 = 1'.

4. If there is carry from column 1 to 2 then 'N+R = E mod 10' and 'N = E+1' so 'E+1+R = E mod 10', so 'R = 9' but 'S = 9', so there must be carry from column 1 to 2. Therefore 'C_1 = 1' and 'R = 8'.

5. To produce carry 'C_1=1' from column 1 to 2, we must have 'D+E = 10+Y' as Y cannot be 0/1 so D+E is at least 12. As D is at most 7 and E is at least 5(D cannot be 8 or 9 as it is already assigned). N is atmost 7 and 'N = E+1' so 'E = 5 or 6'.

6. If E were 6 and D+E atleast 12 then D would be 7, but 'N = E+1' and N would also be 7 which is impossible. Therefore 'E = 5' and 'N = 6'.

7. D + E is atleast 12 for that we get 'D=7' & 'Y=2'.

```
  9 5 6 7
+ 1 0 8 5
---------
1 0 6 5 2
```

Values :

S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2

ii.

```
    T W O
  + T W O
---------
= F O U R
```


A CSP model is as follows :

- Variables : The letters
- Possible values : {0, 1, ..., 9}
- Constraints : All letters have different values, addition works as intended, leading digits aren't zero.

"Addition works as intended" can be made concrete with these equations, where C_1 and C_{10} are the carry values.

$$O + O = R + 10 * C_1$$

$$W + W + C_1 = U + 10 * C_{10}$$

$$T + T + C_{10} = O + 10 * F$$

"All different" and the equational constraints involve multiple variables. Usually best to keep them in that form, rather than converting to binary. We can visualize this using a slightly different type of graph with some extra nodes :

- Nodes for the basic variables
- Nodes for auxiliary variables (carry values)
- Square boxes for constraints, attached to variables they involve

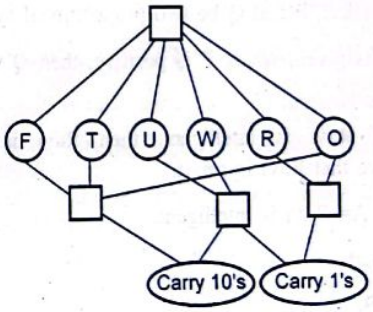


Fig. 2

iii. Refer Q.18 of Chapter - 3.

Q.3 a) *List the inference rules used in prepositional logic? Explain them in detail with suitable example.* **[9]**

Ans. : Inference rules are those rules which are used to describe certain conclusions. The inferred conclusions lead to the desired goal state.

In propositional logic, there are various inference rules which can be applied to prove the given statements and conclude them.

There are following laws/rules used in propositional logic :

i. **Modus tollen :** Let, P and Q be two propositional symbols :

Rule : Given, the negation of Q as (~Q).

If P?Q, then it will be (~P), i.e., the negation of P.

Example : If Anil goes to the temple, then Anil is a religious person. Anil is not a religious person. Prove that Anil doesn't go to temple.

Solution : Let, P = Anil goes to temple.

Q = Anil is religious. Therefore, (~Q) = Anil is not a religious person.

To prove : ~P?~Q

By using **Modus Tollen** rule, P?Q, i.e., ~P?~Q (because the value of Q is (~Q)).

Therefore, Anil doesn't go to the temple.

ii. **Modus Ponen :** Let, P and Q be two propositional symbols :

Rule : If P?Q is given, where P is positive, then Q value will also be positive.

Example : If Ravi is intelligent, then Ravi is smart. Ravi is intelligent. Prove that Ravi is smart.

Solution : Let, A= Ravi is intelligent.

B = Ravi is smart.

To prove : A?B.

By using **Modus Ponen** rule, A?B where A is positive. Hence, the value of B will be true. **Therefore, Ravi is smart.**

iii. **Syllogism :** It is a type of logical inference rule which concludes a result by using deducting reasoning approach.

Rule : If there are three variables say P, Q, and R where

P?Q and Q?R then P?R.

Example : Given a problem statement :

If Raju is the friend of Sanju and Sanju is the friend of Amir, then Raju is the friend of Amir.

Solution : Let, P= Raju is the friend of Sanju.

Q = Sanju is the friend of Amir.

R = Raju is the friend of Amir.

It can be represented as : **If (P?Q) ? (Q?R) = (P?R).**

iv. **Disjunctive Syllogism**

Rule : If (~P) is given and (P V Q), then the output is Q.

Example : Hari is not smart or he is obedient.

Solution : Let, (~P)= Hari is smart.

Q = He is obedient.

P = Hari is not smart.

It can be represented as (P V Q) which results Hari is obedient.

v. **Biconditional elimination :** If Aó B then (A?B) ? (B?A) or

If (A?B) ? (B?A) then A óB. (using one side implication rule).

vi. **Contrapositive :** If (A óB) then ((A?B) ? (B?A))

b) *Explain syntax and semantics of first order logic in detail.* **(Refer Q.9 of Chapter - 5)** **[8]**

OR

Q.4 a) *Detail the algorithm for deciding entailment in prepositional logic.* **(Refer Q.6 of Chapter - 4)** **[8]**

b) *Explain knowledge representation structure and compare them.* **[9]**

Ans. : Knowledge representation in Artificial Intelligence is making machines identify things with the knowledge like a human in understanding, interpreting and analyzing. Below are commonly used knowledge representation structures widely used in AI.

1. Logical representation

Logical representation is a language with some **definite rules** which deal with propositions and has no ambiguity in representation. It represents a conclusion based on various conditions and lays down some important **communication rules.** Also, it consists of precisely defined syntax and semantics which supports the sound inference. Each sentence can be translated into logics using syntax and semantics.

Benefits of logical representation technique

- It helps in performing logical reasoning in knowledge representation.
- It is the fundamental for programming languages.

Limitations of logical representation technique

- It has some restrictions and it is very challenging to work on this.
- It may not be natural and inferences will not be efficient enough for real - world use cases.

2. Production rules

Production rules are a system in itself. It consists of a rule applier, a set of rules, and a database (memory). Whenever an input is passed through, the condition is checked through the production rules and an appropriate rule is selected. The action is carried out based on the rules mentioned.

The whole cycle continues for every single input that is brought through the knowledge representation channel. The production rules system is expressed in terms of natural language and hence is used a lot. The only drawback is that sometimes the rule - based system gets inefficient, as some of the rules may still be active.

Benefits of production rules

- Rules can be expressed in natural language.
- Rules are highly modular and easy to update and remove.

Limitations of production rules

- It doesn't show any learning capacities.
- It doesn't save the result of the problem for future use.
- Many rules to be activated during the program execution.
- Inefficient rule-based production system.

3. Semantic network

As the name suggests, this type of representation works with a network of data. In semantic networks, there are two types of relationships. One is the ISA relationship, and the second is the instance relationship. In the network, the blocks define objects, and the edges (or arcs) define the relationships between the blocks. Although semantic networks take more computational time, their use is extensive as the knowledge represented is simple to understand.

Benefits of semantic network representation

- It is natural representation.
- It conveys meaning transparently.
- It is easy and simple to understand.

Limitations of semantic network representation

- It needs more computational time.
- It is Inadequate and no equivalent quantifiers were presented.
- It is not intelligent and depend on the creator of the AI machine.

4. Representation through frames

A frame is a collection of the attributes and the associated values. Frames are also called slot-filler structures. This is because the slots are the attributes and they are filled by the values of those attributes which represent the knowledge in the environment. Frames make the grouping of data and different object values easier. But sometimes, the inference mechanism is challenging to implement or use as it is a quite generalized approach.

Benefits of frame representation

- Programming will be very easier as grouping the relevant data.
- Easy to understand and visualize.
- Easy to include slots for new relations.
- Easy to add default data and look for missing values.

Limitations of frame representations

- This technique can't be easily processed.
- Inference mechanism may not run smoothly.
- It is a very generalized technique.

Q.5 a) *Explain forward and backward chaining. What factors justify whether reasoning is to be done in forward or backward chaining.*
(Refer Q.18 and Q.15 of Chapter - 5) [9]

b) *What are the reasoning patterns in propositional logic ? Explain them in detail.* [9]

Ans. : Reasoning patterns in propositional logic :

There are standard patterns that can be applied to derive chains of conclusions that lead to the desired goal. These pattern of inference are called inference rules.

The concept of monotonicity

1) It says that the set of entailed sentences can only increase as information is added to the knowledge base.
2) Monotonicity means that inference rules can be applied whenever suitable premises are found in the knowledge base. The conclusion of the rule must follow regardless of what else is in the knowledge base.

Inference rules

1) Modus ponens : It is represented as,

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

It means that whenever any sentences of the form $\alpha \Rightarrow \beta$

and α are given then the sentence β can be inferred.

For example -

If two statements,

i) (Signal Pole Ahead $\wedge$ Signal Red) $\Rightarrow$ Stop

ii) (Signal Pole Ahead $\wedge$ Signal Red) are given then

"Stop" can be inferred.

2) And-elimination : It says that from conjunction any conjuncts can be inferred.

It is represented as,

$$\frac{\alpha \wedge \beta}{\alpha}$$

For example -

From the sentence,

(Signal Pole Ahead $\wedge$ Signal Red)

The inference,

"Single Red".

can be drawn.

3) Unit resolution [P $\vee$ Q, $\neg$ Q $\vdash$ P] :

Unit resolution rule takes a clause - a disjunction of literals and a literal and produce a new clause.

Rule is,

$$\frac{l_1 \vee \ldots \vee l\,k,\ m}{l_1 \vee \ldots \vee l_{i-1} \vee l_{i+1} \vee \ldots \vee l\,k}$$

where each l is literal. l_i and m are complementary literals. [It means that one is the negation of other].

4) Resolution : The unit resolution rule can be generalized to the full resolution rule,

$$\frac{l_1 \vee \ldots \vee l_k, m_1 \vee \ldots \vee m_n}{l_1 \vee \ldots \vee l_{i-1} \vee l_{i+1} \vee \ldots \vee l_k \vee m_1 \vee \ldots \vee m_{j-1} \vee m_{j+1} \vee \ldots \vee m_n}$$

where l_i and m_j are complementary literals. If we are dealing only with clauses of length two we could write this as

$$\frac{l_1 \vee l_2, \neg l_2 \vee l_3}{l_1 \vee l_3}$$

That is, resolution takes two clauses and produces a new clause containing all the literals of the two original clauses except the two complementary literals.

It is a single inference rule, that yields a complete inference algorithm when coupled with any complete search algorithm.

Conjunctive normal form

1) The resolution rule applies only to disjunctions of literals, so it would seem to be relevant only to knowledge bases and queries consisting of such disjunctions.
2) Every sentence of propositional logic is logically equivalent to a conjunction of disjunctions of literals.
3) A sentence expressed as a conjunction of disjunctions of literals is said to be in Conjunctive Normal Form (CNF).
4) It is used for representing sentences.

Steps for converting propositional logic sentences into CNF

1) Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$:

   For example -

   Consider following statement,

   $B_{1,1} \quad \Leftrightarrow (P_{1,2} \vee P_{2,1})$

After eliminating $\Leftrightarrow$ in above sentence,

$(B_{1,1} \rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \rightarrow B_{1,1})$

2) Eliminate $\rightarrow$, replacing $\alpha \rightarrow \beta$ with $\neg \alpha \vee \beta$:

   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg (P_{1,2}) \vee P_{2,1} \vee B_{1,1})$

3) CNF requires $\neg$ to appear only in literals, so we "move $\neg$ inwards" by repeated application of the following standard equivalences : -

   $\neg (\neg \alpha) \equiv \alpha$ (clouble - negation elimination)

   $\neg (\alpha \wedge \beta) \equiv (\neg \alpha \vee \neg \beta)$ (De Morgan)

   $\neg (\alpha \vee \beta) \equiv (\neg \alpha \wedge \neg \beta)$ (De Morgan)

   In the example, we require just one application of the last rule.

   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg (P_{1,2} \wedge P_{2,1}) \vee B_{1,1})$

4) Now we have a sentence containing nested $\wedge$ and $\vee$ operators applied to literals. We apply the distributivity law, distributing $\vee$ over $\wedge$ wherever possible.

   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \wedge (\neg P_{2,1} \vee B_{1,1}) \vee B_{1,1})$

   Above sentence is now in CNF.

Resolution rule forms the basis for a family of complete inference procedures.

- **Refutation completeness :**

It means that resolution can always be used to either confirm or refute a sentence but it cannot be used to enumerate true sentences.

Resolution algorithm

1) It takes input as knowledge base [a sentence in propositional logic] and α (it is query - which is a sentence in propositional logic).
2) Its output is true or false means that knowledge base do resolves α. It shows that knowledge base $\models \alpha$, for this it will be shown that $(KB \wedge \neg \alpha)$.

3) Steps are as follows,

i) $(KB \wedge \neg \alpha)$ is first converted in to CNF.

ii) Resolution rule is applied to the resulting clauses.

iii) Each pair that contains complementary literals is resolved to produce new clause.

iv) This new clause is added to set if it is not already present.

v) Step (ii) to (iv) are repeated until one of the two following situations occurs.

a) There are no new clauses that can be added, in which case α does not entail β

OR

b) An application of the resolution rule derives the empty clause, in which case α entails β.

Completeness of resolution

The algorithm of resolution is complete. It means that it will surely produces a result checking all clauses in KB and all the clauses derivable from them. It finds closure set of clauses set which has all derivable clauses from existing clauses set. It checks the input query against the closure set.

Ground resolution theorem

It states that – "If a set of clauses is unsatisfiable, then the resolution closure of those clauses contains the empty clause".

The ground resolution theorem is called as completeness theorem for resolution.

OR

Q.6 a) *Explain unification algorithm with an example.*
(Refer Q.6 of Chapter - 5) [8]

b) *Explain knowledge representation structures and compare them.*
(Refer above Q.4(b) [7]

c) *What do you mean by Ontology of situation calculus ?*
(Refer Q.38 of Chapter - 5) [3]

Q.7 a) *Analyse various planning approaches in detail.*
(Refer Q.16 of Chapter - 6) [9]

b) *Discuss AI and its ethical concerns. Explain limitations of AI.* [8]

Ans. : AI and ethical concerns :

- AI ethics is a system of moral principles and techniques intended to inform the development and responsible use of artificial intelligence technology. As AI has become integral to products and services, organizations are starting to develop AI codes of ethics. An AI code of ethics, also called an AI value platform, is a policy statement that formally defines the role of artificial intelligence as it applies to the continued development of the human race. The purpose of an AI code of ethics is to provide stakeholders with guidance when faced with an ethical decision regarding the use of artificial intelligence. Below are key AI ethics points to note,
  - The potential of automation technology to give rise to job losses
  - The need to redeploy or retrain employees to keep them in jobs
  - Fair distribution of wealth created by machines
  - The effect of machine interaction on human behaviour and attention
  - The need to address algorithmic bias originating from human bias in the data
  - The security of AI systems (eg autonomous weapons) that can potentially cause damage
  - The need to mitigate against unintended consequences, as smart machines are thought to learn and develop independently

AI is a technology designed by humans to replicate, augment or replace human intelligence. These tools typically rely on large volumes of various types of data to develop insights. Poorly designed projects built on data that is faulty, inadequate or biased can have unintended, potentially harmful, consequences. Moreover, the rapid advancement in algorithmic

systems means that in some cases it is not clear to us how the AI reached its conclusions, so we are essentially relying on systems we can't explain to make decisions that could affect society. An AI ethics framework is important because it shines a light on the risks and benefits of AI tools and establishes guidelines for its responsible use. Coming up with a system of moral tenets and techniques for using AI responsibly requires the industry and interested parties to examine major social issues and ultimately the question of what makes us human.

OR

Q.8 a) *Explain the terms for time and schedule from perspective of temporal planning.* [9]

Ans. : • The most basic form of planning views actions as atomic entities that are taken one at a time in a sequence (see classical planning.) When multiple things can be happening at a time, it is necessary to model the *duration* and *concurrency* of actions and events. Multiple actions can be taken simultaneously, their durations may vary and actions and events may have complex interdependencies which determine which combinations are possible. This is called *temporal planning*.

- Closely connected to temporal planning are various *scheduling* problems, where similarly the duration and interdependencies between different events or tasks are central. In scheduling the tasks are given and the core problem to solve is determining how the tasks can be scheduled, i.e. assigned a starting time, so that given resource and ordering constraints are satisfied. Temporal planning involves solving the same scheduling problem, but additionally the choice of actions/tasks is left open and they must be selected together with their scheduling, given a model of the actions and their effects.
- Temporal constraints **describe relationships among variables that refer somehow to time.** A set of temporal constraints can be stored in a temporal database, which is queried by temporal queries during problem solving.

Models of timed state transition systems

- Due to the possibility of multiple concurrent actions, transition systems cannot be modelled as easily as in classical planning as a (finite) set of states together with viewing actions as binary relations on the states.

State variables

- Similarly to classical planning, the state of the world at a given time point is represented as the values of state variables. The values can be changed by actions.

Time-delays in actions

- For a full description of what happens at a given time moment the values of state variables is in general not sufficient. A central feature of temporal models is that actions and events have effects that span a (potentially very long) time interval. For this it is necessary to somehow represent, for every time point, the things that will happen at later time points.
- For example, an action taken at a time point t can have its effects at t or at later time points.

Resources

- Resources represent dependencies between actions. Temporal overlap between two actions is limited if they need the same resource in conflicting ways. For example, two actions that both exclusively require the same machine, tool or device cannot take place at the same time. The main resource types are the following,

1. unary resources : A unary resource may be used by at most one action at a time.
2. n-ary resources : An n-ary resource may be used by at most n actions at a time. Different actions could use different amounts of the resource. So 2 units and 1 unit of a 3-ary resource could be used at a time by two actions, and further actions could not use the resource.
3. State resources : The resource could be viewed to have multiple states, and the resource could not be used in different states at the same time, but the number of actions using the resource in one state would not be limited.

Explicit state-space search

Explicit state-space search, meaning the generation of states reachable from the initial state one by one, is the earliest and most straightforward method for solving some of the most important problems about transition systems, including model-checking (verification), planning, and others, widely used since at least the 1980s.

A timed state consists of

1. An assignment of values to state variables, and
2. A set of clocks, indicating the time remaining for a specific change or event.

- The state variables are as in classical planning. Typically each action is associated with one clock. The clock is initialized to 0 when the action is taken. When a clock reaches a specific value, corresponding changes to the state will take place. For actions this is typically the changes in the state variables that are later than the action's starting point. Additionally, the satisfaction of constraints on the use of resources is defined in terms of the clocks. Specifically, for an action to be possible at a given (timed) state, its precondition has to be true, and the clocks of actions using the same resources have to have values that indicate legal use of shared resources.
- Given a timed state, two types of changes are possible.
    1. Time can be advanced by a given amount t. This is possible if none of the active clocks have critical values > c and < c+t. The state variables retain their old values, but all clocks are advanced from their current value by t.
    2. An action can be taken. This is possible if its precondition and resource constraints are satisfied. The action's clock is set to 0. If the action has immediate (time 0) effects, they will change the values of some state variables. Other state variables remain unchanged.
- Similarly to classical planning, the timed version of explicit state-space search is easy to implement, and can be used in connection with

well-known state-space search algorithms, such as breadth-first, informed search algorithms such as A* and various best-first algorithms as well as stochastic search algorithms.

- The basic issues in the implementation of timed state-space search are the same as with explicit state-space search for untimed (asynchronous) systems, with the exception of choosing how much time should be advanced. All the main techniques for improving explicit state-space search were fully developed by the 1990s, including symmetry and partial order methods, and efficient implementation technology. Explicit state-space search methods are understood very well, and implementation technology is well developed. However, as it is with asynchronous systems, when the number of states is high, the applicability of explicit state-space search methods to timed systems is limited by the necessity to search one state at a time.
- **Constraint-Based Search : SAT, SMT, mixed integer linear programming, constraint programming**
- The constraint-based representations represent a sequence of timed states corresponding to the time points where time is stopped between consecutive advance-time operations. These time points can be called as *steps*. For each step the values of all state variables are represented as propositional variables (Boolean variables) for planning as satisfiability and also numeric variables for satisfiability modulo theory.
- The times associated with steps i can be represented either
    - Explicitly as the absolute times T_i, or
    - Implicitly through the differences Δ_i between steps i – 1 and i.

The delayed effects of an action can be represented either of two ways.

- Delays are presented through explicitly represented clocks, which are initialized to 0 when an action is taken, and which trigger effects when the clock reaches a critical value. Clock increases between steps are expressed with Δ_i or T_i , T_i–1. Constraints to prevent going past a critical clock value are needed.

- Alternatively, a given action entails that for (at least) one of the later steps the time difference to the present step ($T_j - T_i$) is the critical value and the effects take place at that step.
- Frame axioms (representing all possible justifications for changes between consecutive steps) are analogous to the classical planning case.

b) *Write a detailed note on AI Architecture.* [8]

Ans. : AI architecture :

- Any application depends on data, whether it's a traditional application or an intelligent application that draws on artificial intelligence and analytics. Data is a fundamental element of every business and is fundamental to its data and AI architecture. Data is your record of the current state of the business, its history, and the base for predicting what might happen. However, on its own, data doesn't do anything. To realize value from data, you need to do something with it. You must understand it and act on it. Typically, understanding and acting on data requires something other than the data, such as a computer program, a query, or a user, which can be a machine or person.
- AI architecture is based on the same idea to deal with the complexities in analytics and AI. An AI architect must know about the information and infrastructure they're working in as well as the current technology landscape in order to build architecture that will work and adapt both now and in the future.
- The **Artificial Intelligence architect** is like the chief data scientist, planning the implementation of solutions, choosing the right technologies and evaluating the evolution of the architecture as the clients' needs change. The industry is changing at a very rapid speed. Technologies that existed years back are obsolete now. Everyone in the present industry has to adopt the changing technology and get knowledge of new technology.
- It was presumed years back that there will be a day when everything will go online, and we have been through that day. Analysts and researchers have now presumed that by 2025, everything will be on

Artificial Intelligence. The Internet of Things (IoT) will be in demand. Along with Artificial Intelligence, analysts have also recommended people to get accustomed to other technologies like Machine Learning, Data Intelligence, etc.

DECEMBER - 2022 [5926]-242(AI&DS) Solved Paper

Course 2019

Time : $2\frac{1}{2}$ Hours] **[Maximum Marks : 70**

Instructions to the Candidates :

1) Answer four questions Q.1 or Q.2, Q.3 or Q.4, Q.5 or Q.6, Q.7 or Q.8.

2) Figures to the right side indicate full Marks.

3) Assume suitable data, if necessary.

4) Neat diagrams must be drawn wherever necessary.

Q.1 a) *What are the issues that need to be addressed for solving CSP efficiently ? Explain the Solutions to them.*
(Refer Q.1(b) of June 2022 AI (Computer)) [9]

b) *Explain heuristic function that can be used in cutting off search in detail.* [9]

Ans. :

1. An evaluation function returns an estimate of the expected utility of the game from a given position, just as the heuristic functions return an estimate of the distance to the goal.
2. It should be clear that the performance of a game-playing program is dependant on quality of its evaluation function. An inaccurate evaluation function will guide an agent toward positions that turn out to be lost.
3. **Designing evaluation function :**
   a) The evaluation function should order the terminal states in the same way as the true utility function; otherwise, an agent using

it might select suboptimal moves even if it can see ahead all the way to the end of the game.

b) The computation must not take too long !

c) For nonterminal states, the evaluation function should be strongly correlated with the actual chances of winning.

OR

Q.2 a) *Explain alpha-beta tree search and cutoff procedure in detail with an example.* **(Refer Q.6 of Chapter - 3)** **[9]**

b) *Define constraints in CSPs. Explain any two types of Constrains in detail.* **(Refer Q.19 of Chapter - 3)** **[5]**

c) *What are the limitations of game search algorithms ?* **[4]**

Ans. : Imperfect and real-time decisions

The minimax algorithm generates the entire game search space, whereas the alpha-beta algorithm allow us to prune large parts of it. However, alpha-beta still has to search all the way to terminal states for at least a portion of the search space. This depth is usually not practical, because moves must be made in a reasonable amount of time-typically a few minutes at most.

For making search faster we can make a heuristic evaluation function that can be applied to states in the search. It will effectively turn nonterminal nodes into terminal leaves. In other words, the suggestion is to alter minimax or alpha-beta in two ways; the utility function is replaced by a heuristic evaluation function, which gives an estimate of the positions utility, and the terminal test is replaced by a cutoff test that decides when to apply heuristic function.

Cutting off search

1. Cutting off search is simple approach for searching faster.
2. The cutting off search is applied to limit the depth.

   If Cutoff-Test (s, depth) then return E(S) problem in cutting off search. (Where E is evaluation function).
3. Cutoff test might be applied in adverse condition.

4. It may stop search before allowable time.
5. Iterative deepening go until time is elapsed.
6. Quiescent position -

a) If a position looks "dynamic", don't even bother to evaluate it.

b) Instead, do a small secondary search until things calm down. For example - After capturing a piece, things look good, but this would be misleading if opponent was about to capture, right back.

c) In general such factors are called **continuation heuristics.**

d) A **quiescent position** is a position which is unlikely to exhibit wild swings (huge changes) in value in near future.

e) Consider following example -

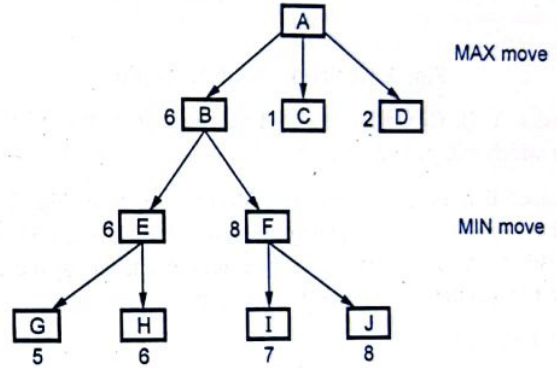


Fig. 1 Quiescent position example

Here [Refer Fig. 1] now since the tree is further explored, the values, which is passed to A, is 6. Thus the situation calms down. This is called as **waiting for quiescence.** This helps in avoiding the **horizon effect of a drastic change of values.**

7. The horizon effect -

A potential problem that arises in game tree search (of a fixed depth) is the horizon effect, which occurs when there is a drastic change in value, immediately beyond the place where the algorithm stops searching. Consider the tree show in Fig. 2 (a).

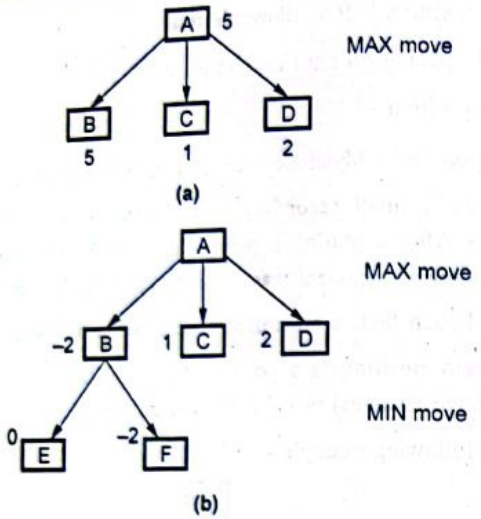


Fig. 2 Horizon effect example

It has nodes A, B, C and D. At this level since it is a maximizing ply, the value which will passed up at A is 5.

Suppose node B is examined one more level as shown in Fig. 2 (b), then we see that because of a minimizing ply, value at B is – 2 and hence the value passed to A is 2. This results in a drastic change in the situation. There are **two proposed solutions** to this problem

a) Singular extension -

Which expands a move that is clearly better than all other moves. Its cost is higher with branching factor 1.

b) Secondary search -

One proposed solution is to examine the search space, beyond the apparently best one, to see that if something is looming just over the horizon. In that case we can revert to second-best move. Obviously then the second-best move has the same problem, and there isn't time to search beyond all possible acceptable moves.

8. **Additional refinements** - Other than alpha-beta pruning, there are a variety of other modifications to the minimax procedure that can also enhance its performance.

During searching, maintain two values alpha and beta so that alpha is the current lower bound of the possible returned value; beta is the current upper bound of the possible returned value. If during searching, we know for sure alpha > beta, then there is no need to search any more in this branch. The returned value cannot be in this branch. Backtrack until it is the case alpha _ beta. The two values alpha and beta are called the ranges of the **current search window.** These values are dynamic. Initially, alpha is – ∞ and beta is ∞.

The quiescence and secondary search technique - If a node represents a state in the middle of an exchange of pieces, the evaluation function may not give a reliable estimate of board quality. For example, after a knight is captured, the evaluation may be good, but this is misleading as the opponent is about to capture your queen.

The solution to this problem is, if node evaluation is not "quiescent", continue alpha-beta search can be made below that node but limit moves to those that significantly change evaluation function (for example, capture moves or promotions). The crucial complexity point is that branching factor for such moves is small.

For complicated games it is not feasible to select a move by simply looking-up the current game configuration in a catalogue **(the book move)** and extracting the current move. The catalogue would be huge and very complex for construction.

Q.3 a) *What are the various approaches to knowledge representation? Explain in detail.* **[9]**

Ans. : A good system for the representation of knowledge in a particular domain should possess the following properties -

- Representational adequacy - It is the ability to represent all of the kinds of knowledge that are needed in that domain.

- Inferential adequacy - It is the ability to manipulate the representational structures in such a way as to derive new structures corresponding to new knowledge inferred from old.
- Inferential efficiency - It is the ability to incorporate into the knowledge structure additional information, that can be used to focus the attention of the inference mechanisms in the most promising direction.
- Acquisitional efficiency - Acquiring new information easily.

Two types of approaches to knowledge representation

1) Simple relational knowledge.

2) Inheritable knowledge.

1) Simple relational knowledge

- This is the simplest way of storing fact which uses relational method, when every and each fact about a set of objects is set out sequentially and automatically in column.
- This type of representation is small procedure for inference.
- It is used to define inference engines.

For example

Player	Weight	Age	Play cricket
Monu	70	30	Right H.
Sonu	65	25	Right H.
Bablee	50	29	Left H.
Soni	45	29	Right H.
Moni	42	25	Left H.

Player_info ('Monu', 70, 30, right H)

2) Inheritable knowledge

- Relational knowledge is made up of object associativity like co-relation associated values attribute.
- All data should be organised into a hierarchy of classes.
- Inherit values from being all members of class.
- Class must be arranged in a generalization.
- Every individual frame can represent the collection of attribute and its value associated with a individual node.

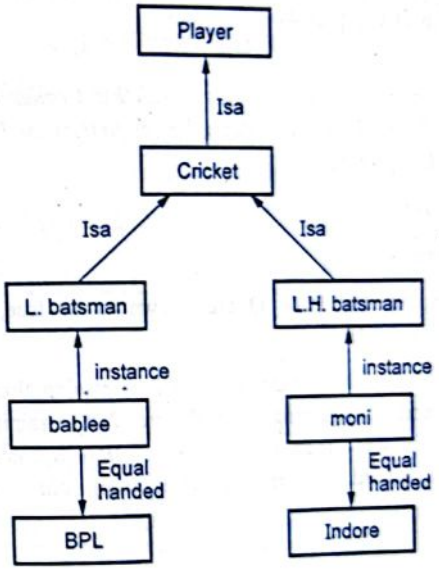


Fig. 3 Inheritable knowledge

Example

Properties of inheritance hierarchy

1) ↕ to be point from object and its value.
2) Boxed : To be object and value of attribute any object.
3) It may be also be called slot-and filter structure.

Algorithm retrieve

To retrieve a value for attribute of an instance object.

1) Find object in the knowledge base.

2) If there is a value for the attribute, report that value.

3) Otherwise look value of instance, if not then fail, otherwise go to the node and find a value for the attribute, if one is found, report it. Otherwise, do until there is no value search using ISA, found for the attribute.

b) *Detail the algorithm for deciding entailment in proposition logic.* **(Refer Q.6 of Chapter 4)** **[8]**

OR

Q.4 a) *Differentiate propositional logic with First order logic. List the Inference rules along with suitable examples for first order logic.* **(Refer Q.24 of Chapter 5)** **[8]**

Ans. : Inference rules :

1) Modus ponens :

If the sentence **P** and **P -> Q** are known to be true, then modus ponens lets us infer **Q**.

For example : If we have statment, " If it is raining then the ground will be wet" and "It is raining". If P denotes " It is raining" and Q is "The ground is wet" then the first expression becomes $P \rightarrow Q$. Because it is indeed now raining (P is true), our set of axioms becomes,

$\{ P \rightarrow Q$

$P \}$

Through an application of modus ponens, the fact that "The ground is wet" (Q) may be added to the set of true expressions.

- **The generalized modus ponens :**
  For atomic sentences P_i, P_i', and q, where there is a substitution Q such that SUBST $(\theta, P_i') =$ SUBST (θ, P_i'),

For all i,

$$\frac{P_1', P_2', \ldots P_n', (P_1 \wedge P_2 \wedge \ldots \wedge P_n \Rightarrow q)}{\text{SUBST}(\theta, q)}$$

There are n+1 premises to this rule : - The 'n' atomic sentences P_i' and the one implication. The conclusion is the result applying the substitution θ to the consequent q.

P_1' is - Princess (Cindrella) P_1 is - Princess (x)

P_2' is - Kind (y) P_2 is - Kind (x)

θ is {x/Cindrella, y/cindrella} q is GoodHearted (x)

SUBST (θ, q) is, GoodHearted (cinderella).

2) Modus tollens :

Under the inference rules modus tollens, if $P \rightarrow Q$ is known to be true and **Q** is known to be false, we can infer $\neg P$.

For example : Given that,

i) $\underbrace{\text{Engine Starts} \wedge \neg \text{ Flat Tire}}_{P} \Rightarrow \underbrace{\text{Car OK}}_{Q}$

ii) $\underbrace{\neg \text{ Car OK}}_{Q}$

Then by Modus tollens,

$\underbrace{\neg (\text{Engine Starts} \wedge \neg \text{ Flat Tire})}_{\neg P}$ which is equivalence to

$\neg$ [Engine Starts $\vee$ FlatTire]

3) And elimination :

And elimination allows us to infer the truth of either of the conjuncts from the truth of a conjunctive sentence. For instance, $P \wedge Q$ is true. Lets us conclude that **P** and **Q** are true.

4) And introduction :

And introduction lets us infer the truth of a conjunction from the truth of its conjuncts. For instance, if P and Q are true, then $P \wedge Q$ is true.

5) Inference rules for quantifiers :

1. Universal Instantiation (UI) :

This rule states that, we can infer any sentence obtained by substituting a ground term (a term without variable) for the variable.

Formally, this rule is described with the concept of substitution.

Substitution :

SUBST (θ, S) denotes the result of applying the substitution θ to the sentence S.

With the substitution we can write UI rule as follows,

$$\frac{\forall\ v\ S}{\text{SUBST}(\{v/g\}, S)}$$

For any variable V, ground term g.

UI can be applied many times to produce many different consequences.

For example :

All beautiful princess are GoodHearted.

$\forall$ x Princess (x) $\wedge$ Beautiful (x) $\Rightarrow$ GoodHearted (x)

From above axioms we can infer following permissible sentences : -

i. Princess (Seeta) $\wedge$ Beautiful (Seeta) $\Rightarrow$ GoodHearted (Seeta)

ii. Princess (Indumati) $\wedge$ Beautiful (Indumati) $\Rightarrow$ GoodHearted (Indumati)

iii. Princess (Mother (Seeta)) $\wedge$ Beautiful (Mother (Seeta)) $\Rightarrow$ Good Hearted (Mother (Seeta))

2. Existential Instantiation (EI)

This rule states that, "For any sentence S, variable V and constant symbol k that does not appear else where in the KB, following statement holds,

$$\frac{\exists\ V, S}{\text{SUBST}(\{V/k\}, S)}$$

Basically the existential statement say that, there is some object satisfying a condition. The instantiation process is just giving a name to this object. [It should be noted that this name must not already belong to another object].

For example :

$\exists$ x Crown (x) $\wedge$ OnHead (x, Cindrella)

We can infer sentence,

Crown (H) $\wedge$ On Head (H, Cindrella) as long as H does not appear else where in the knowledge base.

Existential Instatiation can be applied once and then existentially quantified sentence can be discarded.

For example :

If we add sentence,

Arrested (Police, Thief) then we do not need,

$\exists$ x Arrested (x, Thief)

b) *Explain knowledge representation structures and compare them.* **(Refer Q.4 (b) of June-2022 AI (Computer))** **[9]**

Q.5 a) *Explain Unification algorithm with suitable example.* **(Refer Q.6 of Chapter 5)** **[9]**

b) *What is knowledge engineering ? Explain ontology of situation calculus.* **(Refer Q.36 and Q.38 of Chapter - 5)** **[9]**

OR

Q.6 a) *Explain the forward chaining process and efficient forward chaining with example. State its usage.* **(Refer Q.18 of Chapter - 5)** **[8]**

b) *What are the reasoning patterns in propositional logic ? Explain them in detail.* [7]

Ans. : There are standard patterns that can be applied to derive chains of conclusions that lead to the desired goal. These pattern of inference are called inference rules.

The concept of monotonicity

1) It says that the set of entailed sentences can only increase as information is added to the knowledge base.

2) Monotonicity means that inference rules can be applied whenever suitable premises are found in the knowledge base. The conclusion of the rule must follow regardless of what else is in the knowledge base.

Inference rules

1) Modus Ponens

It is represented as,

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

It means that whenever any sentences of the form $\alpha \Rightarrow \beta$

and α are given then the sentence β can be inferred.

For example -

If two statements,

i) (Signal Pole Ahead $\wedge$ Signal Red) $\Rightarrow$ Stop

ii) (Signal Pole Ahead $\wedge$ Signal Red) are given then

"Stop" can be inferred.

2) And-elimination :

It says that from conjunction any conjuncts can be inferred.

It is represented as,

$$\frac{\alpha \wedge \beta}{\alpha}$$

For example -

From the sentence,

(Signal Pole Ahead $\wedge$ Signal Red)

The inference,

"Single Red".

can be drawn.

3) Unit resolution [P $\vee$ Q, $\neg$ Q $\vdash$ P] :

Unit resolution rule takes a clause - a disjunction of literals and a literal and produce a new clause.

Rule is,

$$\frac{l_1 \vee \ldots \vee l\,k,\ m}{l_1 \vee \ldots \vee l_{i-1} \vee l_{i+1} \vee \ldots \vee l k}$$

where each l is literal. l_i and m are complementary literals. [It means that one is the negation of other].

4) Resolution :

The unit resolution rule can be generalized to the full resolution rule,

$$\frac{l_1 \vee \ldots \vee l_k, m_1 \vee \ldots \vee m_n}{l_1 \vee \ldots \vee l_{i-1} \vee l_{i+1} \vee \ldots \vee l_k \vee m_1 \vee \ldots \vee m_{j-1} \vee m_{j+1} \vee \ldots \vee m_n}$$

where l_i and m_j are complementary literals. If we are dealing only with clauses of length two we could write this as

$$\frac{l_1 \vee l_2, \neg l_2 \vee l_3}{l_1 \vee l_3}$$

That is, resolution takes two clauses and produces a new clause containing all the literals of the two original clauses except the two complementary literals.

It is a single inference rule, that yields a complete inference algorithm when coupled with any complete search algorithm.

Conjunctive normal form

1) The resolution rule applies only to disjunctions of literals, so it would seem to be relevant only to knowledge bases and queries consisting of such disjunctions.
2) Every sentence of propositional logic is logically equivalent to a conjunction of disjunctions of literals.
3) A sentence expressed as a conjunction of disjunctions of literals is said to be in Conjunctive Normal Form (CNF).
4) It is used for representing sentences.

Steps for converting propositional logic sentences into CNF

1) Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$:

   For example -

   Consider following statement,

   $$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

   After eliminating $\Leftrightarrow$ in above sentence,

   $$(B_{1,1} \rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \rightarrow B_{1,1})$$

2) Eliminate $\rightarrow$, replacing $\alpha \rightarrow \beta$ with $\neg \alpha \vee \beta$:

   $$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg (P_{1,2}) \vee P_{2,1} \vee B_{1,1})$$

3) CNF requires $\neg$ to appear only in literals, so we "move $\neg$ inwards" by repeated application of the following standard equivalences : -

   $\neg (\neg \alpha) \equiv \alpha$ (clouble - negation elimination)

   $\neg (\alpha \wedge \beta) \equiv (\neg \alpha \vee \neg \beta)$ (De Morgan)

   $\neg (\alpha \vee \beta) \equiv (\neg \alpha \wedge \neg \beta)$ (De Morgan)

In the example, we require just one application of the last rule.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg (P_{1,2} \wedge P_{2,1}) \vee B_{1,1})$$

4) Now we have a sentence containing nested $\wedge$ and $\vee$ operators applied to literals. We apply the distributivity law, distributing $\vee$ over $\wedge$ wherever possible.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \wedge (\neg P_{2,1} \vee B_{1,1}) \vee B_{1,1})$$

Above sentence is now in CNF.

Resolution rule forms the basis for a family of complete inference procedures.

- **Refutation completeness :**

It means that resolution can always be used to either confirm or refute a sentence but it cannot be used to enumerate true sentences.

Resolution algorithm

1) It takes input as knowledge base [a sentence in propositional logic] and α (it is query - which is a sentence in propositional logic).
2) Its output is true or false means that knowledge base do resolves α. It shows that knowledge base $\models \alpha$, for this it will be shown that $(KB \wedge \neg \alpha)$.
3) Steps are as follows,
   i) $(KB \wedge \neg \alpha)$ is first converted in to CNF.
   ii) Resolution rule is applied to the resulting clauses.
   iii) Each pair that contains complementary literals is resolved to produce new clause.
   iv) This new clause is added to set if it is not already present.
   v) Step (ii) to (iv) are repeated until one of the two following situations occurs.

a) There are no new clauses that can be added, in which case α does not entail β

OR

b) An application of the resolution rule derives the empty clause, in which case α entails β.

Completeness of resolution

The algorithm of resolution is complete. It means that it will surely produces a result checking all clauses in KB and all the clauses derivable from them. It finds closure set of clauses set which has all derivable clauses from existing clauses set. It checks the input query against the closure set.

Ground resolution theorem

It states that – "If a set of clauses is unsatisfiable, then the resolution closure of those clauses contains the empty clause".

The ground resolution theorem is called as completeness theorem for resolution.

c) *Write a note on : Categories and objects.*
(Refer Q.37 of Chapter - 5) [3]

Q.7 a) *Explain time, schedules and resources in temporal domain with an example.* [9]

Ans. : • The most basic form of planning views actions as atomic entities that are taken one at a time in a sequence (see classical planning.) When multiple things can be happening at a time, it is necessary to model the *duration* and *concurrency* of actions and events. Multiple actions can be taken simultaneously, their durations may vary and actions and events may have complex interdependencies which determine which combinations are possible. This is called *temporal planning*.

• Closely connected to temporal planning are various *scheduling* problems, where similarly the duration and interdependencies between different events or tasks are central. In scheduling the tasks are given and the core problem to solve is determining how the tasks can be scheduled, i.e. assigned a starting time, so that given resource and ordering constraints are satisfied. Temporal planning involves solving the same scheduling problem, but additionally the choice of

actions/tasks is left open and they must be selected together with their scheduling, given a model of the actions and their effects.

• Temporal constraints **describe relationships among variables that refer somehow to time.** A set of temporal constraints can be stored in a temporal database, which is queried by temporal queries during problem solving.

Models of timed state transition systems

• Due to the possibility of multiple concurrent actions, transition systems cannot be modelled as easily as in classical planning as a (finite) set of states together with viewing actions as binary relations on the states.

State variables

• Similarly to classical planning, the state of the world at a given time point is represented as the values of state variables. The values can be changed by actions.

Time-delays in actions

• For a full description of what happens at a given time moment the values of state variables is in general not sufficient. A central feature of temporal models is that actions and events have effects that span a (potentially very long) time interval. For this it is necessary to somehow represent, for every time point, the things that will happen at later time points.

• For example, an action taken at a time point t can have its effects at t or at later time points.

Resources

• Resources represent dependencies between actions. Temporal overlap between two actions is limited if they need the same resource in conflicting ways. For example, two actions that both exclusively require the same machine, tool or device cannot take place at the same time. The main resource types are the following,

1. unary resources : A unary resource may be used by at most one action at a time.

2. n-ary resources : An n-ary resource may be used by at most n actions at a time. Different actions could use different amounts of the resource. So 2 units and 1 unit of a 3-ary resource could be used at a time by two actions, and further actions could not use the resource.

3. State resources : The resource could be viewed to have multiple states, and the resource could not be used in different states at the same time, but the number of actions using the resource in one state would not be limited.

Explicit state-space search

Explicit state-space search, meaning the generation of states reachable from the initial state one by one, is the earliest and most straightforward method for solving some of the most important problems about transition systems, including model-checking (verification), planning, and others, widely used since at least the 1980s.

A timed state consists of

1. an assignment of values to state variables, and

2. a set of clocks, indicating the time remaining for a specific change or event.

- The state variables are as in classical planning. Typically each action is associated with one clock. The clock is initialized to 0 when the action is taken. When a clock reaches a specific value, corresponding changes to the state will take place. For actions this is typically the changes in the state variables that are later than the action's starting point. Additionally, the satisfaction of constraints on the use of resources is defined in terms of the clocks. Specifically, for an action to be possible at a given (timed) state, its precondition has to be true, and the clocks of actions using the same resources have to have values that indicate legal use of shared resources.
- Given a timed state, two types of changes are possible.

1. Time can be advanced by a given amount t. This is possible if none of the active clocks have critical values > c and < c+t.

The state variables retain their old values, but all clocks are advanced from their current value by t.

2. An action can be taken. This is possible if its precondition and resource constraints are satisfied. The action's clock is set to 0. If the action has immediate (time 0) effects, they will change the values of some state variables. Other state variables remain unchanged.

- Similarly to classical planning, the timed version of explicit state-space search is easy to implement, and can be used in connection with well-known state-space search algorithms, such as breadth-first, informed search algorithms such as A* and various best-first algorithms as well as stochastic search algorithms.
- The basic issues in the implementation of timed state-space search are the same as with explicit state-space search for untimed (asynchronous) systems, with the exception of choosing how much time should be advanced. All the main techniques for improving explicit state-space search were fully developed by the 1990s, including symmetry and partial order methods, and efficient implementation technology. Explicit state-space search methods are understood very well, and implementation technology is well developed. However, as it is with asynchronous systems, when the number of states is high, the applicability of explicit state-space search methods to timed systems is limited by the necessity to search one state at a time.
- **Constraint-Based Search : SAT, SMT, mixed integer linear programming, constraint programming**
- The constraint-based representations represent a sequence of timed states corresponding to the time points where time is stopped between consecutive advance-time operations. These time points can be called as *steps*. For each step the values of all state variables are represented as propositional variables (Boolean variables) for planning as satisfiability and also numeric variables for satisfiability modulo theory.
- The times associated with steps i can be represented either

- explicitly as the absolute times T_i, or
- implicitly through the differences Δ_i between steps i-1 and i.

The delayed effects of an action can be represented either of two ways.

- Delays are presented through explicitly represented clocks, which are initialized to 0 when an action is taken, and which trigger effects when the clock reaches a critical value. Clock increases between steps are expressed with Δ_i or T_i , T_i-1. Constraints to prevent going past a critical clock value are needed.
- Alternatively, a given action entails that for (at least) one of the later steps the time difference to the present step ($T_j - T_i$) is the critical value and the effects take place at that step.
- Frame axioms (representing all possible justifications for changes between consecutive steps) are analogous to the classical planning case.

b) *Discuss AI and its ethical concerns. Explain Limitations ofAI.* **[8]**

Ans. : • AI ethics is a system of moral principles and techniques intended to inform the development and responsible use of artificial intelligence technology. As AI has become integral to products and services, organizations are starting to develop AI codes of ethics. An AI code of ethics, also called an AI value platform, is a policy statement that formally defines the role of artificial intelligence as it applies to the continued development of the human race. The purpose of an AI code of ethics is to provide stakeholders with guidance when faced with an ethical decision regarding the use of artificial intelligence. Below are key AI ethics points to note,

- The potential of automation technology to give rise to job losses
- The need to redeploy or retrain employees to keep them in jobs
- Fair distribution of wealth created by machines
- The effect of machine interaction on human behaviour and attention
- The need to address algorithmic bias originating from human bias in the data

- The security of AI systems (eg autonomous weapons) that can potentially cause damage
- The need to mitigate against unintended consequences, as smart machines are thought to learn and develop independently

• AI is a technology designed by humans to replicate, augment or replace human intelligence. These tools typically rely on large volumes of various types of data to develop insights. Poorly designed projects built on data that is faulty, inadequate or biased can have unintended, potentially harmful, consequences. Moreover, the rapid advancement in algorithmic systems means that in some cases it is not clear to us how the AI reached its conclusions, so we are essentially relying on systems we can't explain to make decisions that could affect society. An AI ethics framework is important because it shines a light on the risks and benefits of AI tools and establishes guidelines for its responsible use. Coming up with a system of moral tenets and techniques for using AI responsibly requires the industry and interested parties to examine major social issues and ultimately the question of what makes us human.

OR

Q.8 a) *Analyze various planning approaches in detail.*
(Refer Q.16 of Chapter - 6) **[9]**

b) *Explain AI architecture with a suitable diagram.* **[8]**

Ans. : • Any application depends on data, whether it's a traditional application or an intelligent application that draws on artificial intelligence and analytics. Data is a fundamental element of every business and is fundamental to its data and AI architecture. Data is your record of the current state of the business, its history, and the base for predicting what might happen. However, on its own, data doesn't do anything. To realize value from data, you need to do something with it. You must understand it and act on it. Typically, understanding and acting on data requires something other than the data, such as a computer program, a query, or a user, which can be a machine or person.

• AI architecture is based on the same idea to deal with the complexities in analytics and AI. An AI architect must know about the information

and infrastructure they're working in as well as the current technology landscape in order to build architecture that will work and adapt both now and in the future.

- The **Artificial Intelligence architect** is like the chief data scientist, planning the implementation of solutions, choosing the right technologies and evaluating the evolution of the architecture as the clients' needs change. The industry is changing at a very rapid speed. Technologies that existed years back are obsolete now. Everyone in the present industry has to adopt the changing technology and get knowledge of new technology.
- It was presumed years back that there will be a day when everything will go online, and we have been through that day. Analysts and researchers have now presumed that by 2025, everything will be on Artificial Intelligence. The Internet of Things (IoT) will be in demand. Along with Artificial Intelligence, analysts have also recommended people to get accustomed to other technologies like Machine Learning, Data Intelligence, etc.

DECEMBER - 2022 [5926]-67(COMP.) Solved Paper

Course 2019

Time : 2 $\frac{1}{2}$ Hours] **[Maximum Marks : 70**

Q.1 a) *Explain min max and alpha beta pruning algorithm for adversarial search with example.* **(Refer Q.4 and Q.6 of Chapter - 3) [9]**

b) *Define and explain constraints satisfaction problem.* **(Refer Q.19 of Chapter - 3) [9]**

OR

Q.2 a) *Explain with example graph coloring problem.* **(Refer Q.19 of Chapter - 3) [9]**

b) *How AI technique is used to solve tic-tac-toe problem.* **(Refer Q.11 of Chapter - 3) [9]**

Q.3 a) *Explain Wumpus world environment giving its PEAS description.* **[9]**

Ans. : A wumpus world agent : Wumpus world is a classic artificial intelligence problem, which is used to demonstrate various aspects based on simulation, as well as other AI concepts.

Wumpus was an early computer game in which an agent had to explore a cave made up from a series of interconnected rooms. In one of the rooms in the cave, there was a Wumpus which would kill the agent if it entered that room. Some rooms contained pits, and the agent would die if it entered any of those rooms too. The agent had one arrow with which it could kill the Wumpus. The goal was to locate the gold that was hidden some where in the cave and return to the start without getting killed.

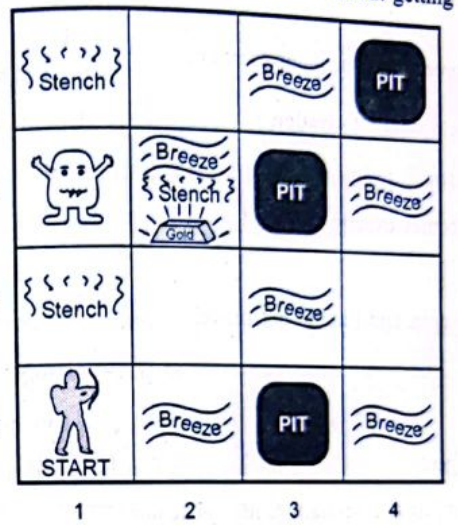


Fig. 1 A wumpus world agent and its environment

PEAS description :

- **Performance measure :**
  Gold : + 1000, **Death :** – 1000
  – 1 per step, – 10 for using the arrow.
- **Environment :**
  - Squares adjacent to Wumpus are **smelly.**
  - Squares adjacent to pit are **breezy.**

- **Glitter** iff gold is in the same square.
- Shooting kills Wumpus if you are facing it. It **screams**.
- Shooting uses only arrow.
- Grabbing picks up gold if in same square.
- Releasing drops the gold in same square.
- You **bump** if you walk into a wall.

• **Actuators :**
Left turn, right turn, forward, grab, release, shoot.

• **Sensors :**
Stench, breeze, glitter, bump, scream.

Wumpus world characterization :

1) Deterministic

Yes - outcomes exactly specified.

2) Static

Yes - Wumpus and Pits do not move.

3) Discrete

Yes

4) Single - agent

Yes - Wumpus is essentially a natural feature.

5) Fully observable

No - only **Local** perception.

Exploring the wumpus world :

1) The knowledge base initially contains the rules of the environment.
2) Location : [1, 1]

Percept : [¬ Stench, ¬ Breeze, ¬ Glitter, ¬ Bump, ¬ Scream]

Action : Move to safe cell [2, 1].

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

A = Agent
B = Breeze
G = Glitter gold
OK = Soft square
P = Pit
S = Stench
V = Visited
W = Wumpus

Fig. 2 Exploring wumpus world-I

3) Location : [2, 1]

Percept : [¬ Stench, Breeze, ¬ Glitter, ¬ Bump, ¬ Scream].

Infer : Breeze indicates that there is a pit in [2, 2] or [3, 1].

Action : Return to [1, 1] to try next safe cell.

4) Location : [1, 2]

Percept : [Stench, ¬ Breeze, ¬ Glitter, ¬ Bump, ¬ Scream].

Infer : Wumpus in [1, 3] or [2, 2]

YET ..... not in [1, 1]

Thus .... not in [2, 2] or stench would have been detected in [2, 1]

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

A = Agent
B = Breeze
G = Glitter gold
OK = Soft square
P = Pit
S = Stench
V = Visited
W = Wumpus

Fig. 3 Exploring wumpus world - II

Thus .... Wumpus is in [1, 3] [2, 2] is safe because of lack of breeze in [1, 2]

Thus .... pit in [3, 1]

Action : Move to next safe cell [2, 2].

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

A = Agent
B = Breeze
G = Glitter gold
OK = Soft square
P = Pit
S = Stench
V = Visited
W = Wumpus

Fig. 4 Exploring wumpus world - III

b) *Explain different inference rules in FOL with suitable example.* **(Refer Q.4(a) of Dec.-2022 AI-AIDS)** **[8]**

OR

Q.4 a) *Write an propositional logic for the statement.* **[10]**

i) "All birds fly"

ii) "Every man respect his parents" **(Refer Q.4 of Chapter - 5)**

b) *Differentiate between propositional logic and first order logic.* **(Refer Q.24 of Chapter - 5)** **[7]**

Q.5 a) *Explain forward chaining algorithm with the help of example.* **(Refer Q.18 of Chapter - 5)** **[9]**

b) *Write and explain the steps of knowledge engineering process.* **(Refer Q.13 of Chapter - 5)** **[9]**

OR

Q.6 a) *Explain backward chaining algorithm with the help of example* **(Refer Q.18 of Chapter - 5)** **[9]**

b) *Write a short note on :*

i) Resolution and ii) Unification **(Refer Q.6 of Chapter - 5)** **[9]**

Ans. : i) Resolution : It is procedure of inferencing the given statement based on available data. The resolution procedure we are going to study is the lifted version of resolution procedure in propositional logic.

In first order logic, for resolution, it requires the sentences in Conjunctive Normal Form (CNF). That is, a conjunction of clauses, where each clauses is a disjunction of literals. Literals can contain variables, which are assumed to be universally quantified.

- General format of conjunctive normal form with exactly K literals per clause is

  $(l_{1,1} \vee .... \vee l_{1,K}) \wedge .... \wedge (l_{n,1} \vee ..... \vee l_{n,K})$

- Generally there can be any number of literals in a single clause.
  For example :

  $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

- CNF statement with quantifiers -
  $[\forall x\ A(x) \vee \forall y\ B(y)] \wedge [\exists z\ C(z)]$

- Consider another example :
  $\forall x$ **American (x)** $\wedge$ **Weapon (y)** $\wedge$ **Sells (x, y, z)** $\wedge$ **Hostile (z)** $\Rightarrow$ **Criminal (x)**

  The above sentence in CNF becomes,

  $\neg$ **American (x)** $\vee$ $\neg$ **Weapon (y)** $\vee$

  $\neg$ **Sells (x, y, z)** $\vee$ $\neg$ **Hostile (z)** $\vee$ **Criminal (x)**

- Note that,
  CNF sentence will be unsatisfiable only when original sentences is unsatisfiable Therefore we will have resolution proofs by contradiction on the CNF sentences.

- **The general steps for converting first order logic sentences to conjunctive normal form**

1. $(\forall x)(P(x)) \Rightarrow ((\forall y)(P(y) \Rightarrow P(F(x,y))) \wedge \neg(\forall y)(Q(x,y) \Rightarrow P(y)))$(like $A \Rightarrow B \wedge C$)

2. Eliminate $\Rightarrow$

$(\forall x)(\neg P(x) \vee ((\forall x)(\neg P(y) \vee P(F(x,y))) \wedge \neg(\forall y)(\neg Q(x,y) \vee P(y)))$

3. Reduce scope of negation :

$(\forall x)\ (\neg P(x) \vee ((\forall y)\ (\neg P(y) \vee P\ (F(x, y))) \wedge (\exists y)\ (Q(x, y) \wedge \neg P(y))))$

4. Standardize variables :

$(\forall x)\ (\neg P(x) \vee ((\forall y)\ (\neg P(y) \vee P(F(x, y))) \wedge (\exists z)\ (Q(x, z) \wedge \neg P(z))))$

$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee (\neg P(y) \vee P(F(x,y))) \wedge (\exists z)(Q(x,z) \wedge \neg P(z))))$

5. Eliminate existential quantification.

(Skolemization) [The detail procedure is explained next]

$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(F(x,y))) \wedge (Q(x, g(x)) \wedge \neg P(g(x)))$

6. Drop universal quantification symbols :

$(\neg P(x) \vee ((\neg P(y) \vee P(F(x, y))) \wedge (Q(x, g(x)) \wedge \neg P(g(x)))))$

7. Convert to conjunction of disjunctions :

$(\neg P(x) \vee \neg P(y) \vee P\ (F(x, y))) \wedge (\neg P(x) \vee Q(x, g(x))) \wedge (\neg P(x) \vee \neg P(g(x)))$.

$(\neg P(x) \vee \neg P(y) \vee P\ (F(x, y))) \wedge (\neg P(x) \vee Q(x, g(x))) \wedge (\neg P(x) \vee \neg P(g(x)))P$

8. Create separate clauses :

$\neg P(x) \vee \neg P(y) \vee P(F(x, y))$

$\neg P(x) \vee Q\ (x, g(x))$

$\neg P(x) \vee \neg P(g(x))$.

9. Standardize variables :

$\neg P(x) \vee \neg P(y) \vee P(F(x, y))$

$\neg P(z) \vee Q\ (z, g(z))$

$\neg P(w) \vee \neg P(g(w))$.

- **The resolution inference rule :**

1) The resolution rule for first order clauses is simply a lifted version of the propositional resolution rule.

2) Two clauses, which are assumed to be standardized apart so that they share no variables, can be resolved if they contain complementary literals.

3) First order literals are complementary if one unifies with the negation of the other.

4) The resolution inference rule is

$$\frac{l_1, V \ldots V\, l_k, \qquad m_1 V \ldots V\, m_n}{\text{SUBST}(\theta, l_1, V \ldots V\, l_{i-1}\ V,\ l_{i+1} V \ldots V\, l_k\ V_m, V \ldots V_{mj-1} V_{mj+1} V \ldots V_{m_n})}$$

where UNIFY $(l_i, \neg mj) = \theta$ for example,

we can resolve the two clauses,

[Animal (F(x)) $\vee$ Loves (G(x), x)] and

[$\neg$ Loves (u, v) $\vee$ $\neg$ kills (u, v)]

by eliminating the complementary literals Loves (G(x), x) and $\neg$ Loves (u, v), with unifier.

θ = {u/G(x), v/x}, to produce the resolvent clause.

[Animal (F(x)) $\vee$ $\neg$ kills (G(x), x)]

5) The rule we have just given is the binary resolution rule, because it resolves exactly two literals. The binary resolution rule by itself does not yield a complete inference procedure.

6) The full resolution rule resolves subsets of literals in each clause that are unifiable.

7) Another approach is to extend factoring. Factoring means the removal of redundant literals in first order case. Propositional factoring reduces two literals to one if they are identical; first order factoring reduces two literals to one if they are unifiable.

Q.7 a) *Write a short note on planning agent, state goal and action representation.* **(Refer Q.5 and Q.11 of Chapter - 6)** **[6]**

b) *Explain different components of planning system.* **[6]**

Ans. :

1. Choose the best rule to apply next based on the best available heuristic information.
2. Apply the chosen rule to compute the new problem state that arises from its application.
3. Detect when a solution has been found.
4. Detect when an almost correct solution has been found and employ special techniques to make it totally correct.
5. Detect dead ends so that they can be abandoned and the system's effort directed in more fruitful directions.

1. Choose the rules to apply

- The most widely used technique for selecting an appropriate rules to apply is first to isolate a set of differences between desired goal state and then to identify those rules that are relevant to reducing those differences.
- If several rules, a variety of other heuristic information can be exploited to choose among them.

For example, if we wish to travel by car to visit a friend, then

- The first thing to do is to fill up the car with fuel.
- If we do not have a car then we need to acquire one.
- The largest difference must be tackled first.

2. Applying rules

- In simple systems, applying rules is easy. Each rule simply specified the problem state that would result from its application.
- In complex systems, we must be able to deal with rules that specify only a small part of the complete problem state.
- One way is to describe, for each action, each of the changes it makes to the state description.
- A number of approaches to this task have been used, three of which are discussed below -

(I) Green's Approach (1969)

Basically this states that *we note the changes to a state produced by the application of a rule.* Consider the problem of having two blocks A and B stacked on each other (A on top). Then we may have an initial state S_0 which could be described as :

$$ON(A, B, S_0) \wedge$$

$$ONTABLE(B, S_0) \wedge$$

$$CLEAR(A, S_0)$$

If we now wish to UNSTACK(A, B). We express the operation as follows :

$$[CLEAR(x,s) \wedge ON(x, y, s)] \rightarrow$$

$$[HOLDING(x, DO(UNSTACK(x,y),s)) \wedge$$

$$CLEAR(y,DO(UNSTACK(x,y),s))]$$

where x, y are any blocks, s is any state and DO() specifies that a new state results from the given action.

The result of applying this to state S_0 to give state S_1 then we get :

$$HOLDING(A, S_1)\ CLEAR(B, S_1).$$

There are a three major problems with Green's approach which are as mentioned below

Problem 1 - The frame problem

- In the above we know that *B* is still on the table. This needs to be encoded into *frame axioms* that describe components of the state *not* affected by the operator.

Problem 2 - The qualification problem

- If we resolve the frame problem the resulting description may still be inadequate. Do we need to encode that a block cannot be placed on top of itself ? If so should this attempt *fail* ?

If we allow failure things can get complex - do we allow for a lot of unlikely events ?

Problem 3 - The ramification problem

- After unstacking block *A*, previously, how do we know that *A* is no longer at its initial location ? *Not only is it hard to specify exactly what does not happen (frame problem) it is hard to specify exactly what does happen.*

(II) State description approach

In this approach for each action, a change it makes to the state is described, everything else remains unchanged.

The main **advantage** of this method is single mechanism (that is resolution) can perform all operations that are required for state description. The major **disadvantage** is the number of axioms that are required becomes very large if problem-state description is complex.

(III) STRIPS Approach (1971)

STIPS proposed another approach :

- Basically each operator has three lists of predicates associated with it :
  a list of things that become TRUE called ADD.

  a list of things that become FALSE called DELETE.

  a set of prerequisites that must be true before the operator can be applied.

- Anything not in these lists is assumed to be unaffected by the operation.
- This method initial implementation of STRIPS - has been *extended* to include other forms of reasoning/planning (*e.g.* Nonmonotonic methods, Goal Stack Planning and even Nonlinear planning - We will see later.)
- **Consider the following example in the Blocks World** and the fundamental operations :

STACK

- Requires the arm to be holding a block A and the other block B to be clear. Afterwards the block A is on block B and the arm is empty and these are true - ADD; The arm is not holding a block and block B is not clear; predicates that are false DELETE;

UNSTACK

- Requires that the block A is on block B; that the arm is empty and that block A is clear. Afterwards block B is clear and the arm is holding block A - ADD; The arm is not empty and the block A is not on block B - DELETE;

- We have now greatly reduced the information that needs to be held. If a new attribute is introduced we do not need to add new axioms for existing operators. Unlike in Green's method we remove the state indicator and use a database of predicates to indicate the current state

Thus if the last state was :

ONTABLE(*B*) ^ ON(*A*,*B*) ^ CLEAR(*A*)

after the unstack operation the new state is

ONTABLE(*B*) ^ CLEAR(*B*) ^ HOLDING(*A*) ^ CLEAR(*A*)

3. Detecting progress

The final solution can be detected if,

- We can devise a predicate that is true when the solution is found and is false otherwise.
- Requires a great deal of thought and requires a proof.

4. Detecting a solution

- A planning system has succeeded in finding a solution to a problem when it has found a sequence of operators that transforms the initial problem state into the goal state.
- How will it know when this has been done ?
- In simple problem-solving systems, this question is easily answered by a straightforward match of the state descriptions.
- One of the representative systems for planning systems is, predicate logic. Suppose that as a part of our goal, we have the predicate P(x). To see whether P(x) is satisfied in some state, we ask whether we can prove P(x) given the assertions that describe that state and the axioms that define the world model.

5. Detecting dead ends

- As a planning system is searching for a sequence of operators to solve a particular problem, it must be able to detect when it is exploring a path that can never lead to a solution.
- The same reasoning mechanisms that can be used to detect a solution can often be used for detecting a dead end.
- If the search process is **reasoning forward** from the initial state, it can prune any path that leads to a state from which the goal state cannot be reached.
- If search process is **reasoning backward** from the goal state, it can also terminate a path either because it is sure that the initial state cannot be reached or because little progress is being made. In backward reasoning each goal is decomposed into subgoals. Each of these subgoals may lead to a set of additional subgoals. The major disadvantage is that every subgoal can lead to multiple subgoals making a problem harder than original one.

Detecting false trails is also necessary :

- E.g. A* search - if insufficient progress is made then this trail is aborted in favour of a more hopeful one.

- Sometimes it is clear that solving a problem one way has reduced the problem to parts that are harder than the original state.
- By moving back from the goal state to the initial state it is possible to detect conflicts and any trail or path that involves a conflict can be pruned out.
- Reducing the number of possible paths means that there are more resources available for those left.

Supposing that the computer teacher is ill at a school there are two possible alternatives :

- Ttransfer a teacher from mathematics who knows computing or
- Bring another one in.

Possible problems :

- If the mathematics teacher is the only teacher of mathematics the problem is not solved.
- If there is no money left the second solution could be impossible.

If the problems are nearly decomposable we can treat them as decomposable and then patch them, how ? Consider the final state reached by treating the problem as decomposable at the current state and noting the differences between the goal state and the current state, and the goal state and the initial state and use appropriate Means End analysis techniques to move in the best direction. Better is to work back in the path leading to the current state and see if there are options. It may be that one optional path could lead to a solution whereas the existing route led to a conflict. Generally this means that some conditions are changed prior to taking an optional path through the problem.

Another approach involves putting off decisions until one has to, leaving decision making until more information is available and other routes have been explored. Often some decisions need not be taken as these nodes are never reached.

Repairing an almost correct solution

- The kinds of techniques we are discussing are often useful in solving nearly decomposable problems.

- One good way of solving such problems is to assume that they are completely decomposable, proceed to solve the sub-problems separately, and then check that when the sub-solutions are combined, they do in fact yield a solution to the original problem.

c) *Explain the components of AI.* **[5]**

Ans. : • From Robots to Autonomous cars, Artificial Intelligence 'AI' has been an integral part of our lives. Today, AI has become an essential part of everything big and small, ranging from the self-working manufacturing units to the smallest screens like smartwatches that we use; it is everywhere. Today, companies, independent of their shape or size, rely on Artificial Intelligence to improve their customers' satisfaction and increase sales. Perhaps, it won't be wrong to admit that AI is the next big thing, making its way into the functioning of the fortune 500 companies to allow them to automate their business process.

1. Discover and Learning

- **To discover** is the basic ability of an intelligent system to explore the data from available resources without any human intervention. Then it is processed by the algorithm to explore the large database and automatically finds the relationship between the content and the needed solution to the problem. This not only solves a complex issue but also identify the emergency phenomena.
- Similar to humans, computer programs also learn in different manners. Talking of AI, learning by this platform is further segregated into a varied number of forms. One of the essential **components of AI**, learning for AI includes the trial-and-error method. The solution keeps on solving problems until it comes across the right results. This way, the program keeps a note of all the moves that gave positive results and stores it in its database to use the next time the computer is given the same problem.
- The learning component of AI includes memorizing individual items like different solutions to problems, vocabulary, foreign languages, etc., also known as rote learning. This learning method is later implemented using the generalization method.

b. Reasoning

- The art of reasoning was something that was only limited to humans until five decades ago. The ability to differentiate makes Reasoning one of the essential **components of artificial intelligence**. To reason is to allow the platform to draw inferences that fit with the provided situation. Further, these inferences are also categorized as either inductive or deductive. The difference is that in an inferential case, the solution of a problem provides guarantees of conclusion. In contrast, in the inductive case, the accident is always a result of instrument failure.
- The use of deductive interferences by programming computers has provided them with considerable success. However, reasoning always involves drawing relevant inferences from the situation at hand.

c. Problem-solving

- In its general form, the AI's problem-solving ability comprises data, where the solution needs to find x. AI witnesses a considerable variety of problems being addressed in the platform. The different methods of 'Problem-solving' count for essential **artificial intelligence components** that divide the queries into special and general purposes.
- In the situation of a special-purpose method, the solution to a given problem is tailor-made, often exploiting some of the specific features provided in the case where a suggested problem is embedded. On the other hand, a general-purpose method implies a wide variety of vivid issues. Further, the problem-solving component in AI allows the programs to include step-by-step reduction of difference, given between any goal state and current state.

d. Perception

- In using the 'perception' component of Artificial Intelligence, the element scans any given environment by using different sense-organs, either artificial or real. Further, the processes are maintained internally and allow the perceiver to analyze other scenes in suggested objects and understand their relationship and features. This analysis is often complicated as one, and similar items might pose considerable amounts

of different appearances over different occasions, depending on the view of the suggested angle.

- At its current state, perception is one of those **components of artificial intelligence** that can propel self-driving cars at moderate speeds. FREDDY was one of the robots at its earliest stage to use perception to recognize different objects and assemble different artifacts.

e. Language-understanding

- In simpler terms, language can be defined as a set of different system signs that justify their means using convention. Occurring as one of the widely used **artificial intelligence components**, language understanding uses distinctive types of language over different forms of natural meaning, exemplified overstatements.

OR

Q.8 a) *What are the types of planning ? Explain in detail.* **(Refer Q.7 and Q.12 of Chapter - 6)** **[6]**

b) *Explain classical planning and its advantages with example.* **[6]**

Ans. : Refer Q.1 and Q.9 of Chapter - 6

Advantages of classical planning

- It has provided the facility to develop accurate domain-independent heuristics.
- The systems are easy to understand and work efficiently.
- Classical planning problem has small branching factor there by reducing the time to solve the problem.

c) *Write note on hierarchical task network planning.* **[5]**

Ans. : One can decompose a problem into smaller subproblems (can be called as subtasks) which can be organized as a hierarchical representation. Such hierarchical subproblem representation is called as hierarchical task network. Then we can apply planning to this task network. Such a planning is called as Hierarchical Task Network **(HTN) planning.**

Characteristic of HTN planning

1. In HTN planning the initial plan is very high level description of what is to be done.

   For example : Construction of a building.

2. Representation of action decomposition : -

   Plans are refined by applying action decomposition, as follows -

   i) General descriptions of action decomposition method are stored in a plan library, from where they are retrieved and instantiated to fit the needs of the plan. Each method is expression of the form, **Decompose(a, d)** which means that an action 'a' can be decomposed into plan 'd'.

   ii) The start action of the decomposition supplies all those preconditions of actions in the plan that are not supplied by other actions. These are called as **external preconditions.**

   iii) In, HTN external effects are also represented. These are the preconditions of finish, which are those effects of actions in the plan that are not negated by other actions. External effects can be of two types, namely primary effect (for example → **House**), and secondary effect (for example → ¬ **Money**). Only the primary effects are used for achieving the goals. **The two kinds of effects can conflict with each other.** Such conflicts help to reduce the size of search space.

   iv) **A decomposition should be correct implementation of the action.** A plan 'd' implements an action 'a' correctly, if 'd' is a complete and consistent partial-order plan for the problem, which achieves the effect of 'a' given the preconditions of 'a'.

   v) In HTN, high level preconditions and effects are the **subset of the true preconditions** and **effects of every primitive implementations.**

vi) At high levels in HTN, **some information is hidden.** The high level description completely ignores all internal effects of the decomposition.

For example : Build house decomposition has temporary internal effects, which are permit and contract.

The high-level descriptions do not specify the intervals inside the activity during which the high-level preconditions are effects those must be true. Information hiding reduces complexity in HTN.

Drawback of information hiding is that at high level conflicting actions can not be determine (for example - internal conditions of one high-level action and internal actions of another can conflict) because they are not represented. Due to this, HTN planning algorithm will suffer, while handling conflicts later on.

3. Each action decomposition, **reduces a high-level action to a partially ordered set of lower-level actions.**
4. The process continues until only primitive actions **(actions that the agent can execute automatically)** remain in the plan.
5. The **"Pure" HTN planning** generates plans only by successive action decomposition.
6. HTN planning **uses same notational convention** as in the partial-order planning.

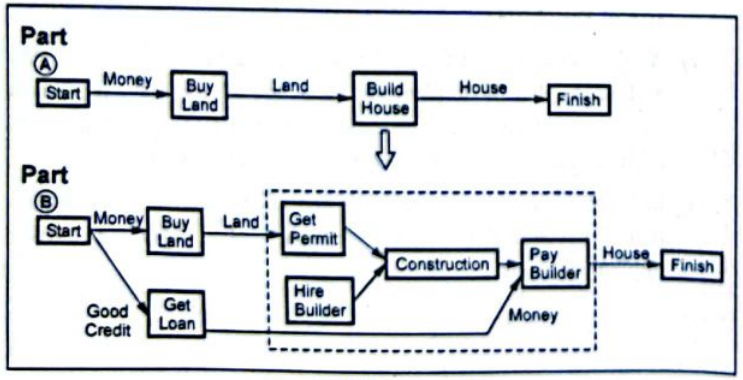


Fig. 5 HTN example

In above diagram,

- Part A shows the existing plan.
- Part B shows the decomposition of a high-level action within an existing plan. The Build House action is replaced by the decomposition, Build House action. The external precondition Land is supplied by the existing causal link from Buy Land. The external precondition Money remains open after the decomposition step, so we add a new action, Get Loan.

8. **Pure HTN planning is undecidable,** even though the underlying state space is finite.
9. The difficulty arises in **pure HTN planning because action decompositions can be recursive.** The HTN can be too long, so that there is no way to terminate the search after any fixed time.
10. To make HTN planning powerful :
    i) One should **rule out recursion** (recursion is really required in very few domains).
    ii) One should **eliminate loops** in HTN that visits same state twice.
    iii) One should **adopt combined approach of POP and HTN** planning. POP surely decides whether plan exists. Therefore, hybrid approach is clearly decidable and solution oriented.

END...